

week7-Applying_Transformations

Macarena López-Mayorga

2026-05-11

First, we attach flowWorkspace and ggcyto to our local environment:

```
library(flowWorkspace)
library(ggcyto)
```

```
StorageLocation <- file.path("C:/Users/Macarena/CytometryInR/course/07_Transformations/data")
fcs_files <- list.files(StorageLocation, ".fcs", full.names=TRUE)
SFC_files <- fcs_files[stringr::str_detect(fcs_files, "2025")]
MC_files <- fcs_files[stringr::str_detect(fcs_files, "G1_2")]
```

Lets start by working with the SFC dataset. First off, let's double check that we have just the SFC files (as flowWorkspace will throw an error if the .fcs files passed contain different number of fluorophore/metal columns)

```
SFC_files
```

```
1_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/2025_07_26_AB_02_INF052_00_0
2_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/2025_07_26_AB_02_INF052_00_0
3_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/2025_07_26_AB_02_INF100_00_0
4_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/2025_07_26_AB_02_INF100_00_0
5_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/2025_07_26_AB_02_INF179_00_0
6_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/2025_07_26_AB_02_INF179_00_0
```

Circling back to where we left off during Week 05, we will pass these file path locations to load_cytoset_from_fcs() to load them into a GatingSet object.

```
SFC_cytoset <- load_cytoset_from_fcs(SFC_files, truncate_max_range=FALSE,
transformation=FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoset)
SFC_GatingSet
```

A GatingSet with 6 samples

One worthwhile thing when first starting is to check the metadata for the GatingSet, using the `pData()` function.

```
pData(SFC_GatingSet)
```

	name
2025_07_26_AB_02_INF052_00_Ctrl.fcs	2025_07_26_AB_02_INF052_00_Ctrl.fcs
2025_07_26_AB_02_INF052_00_SEB.fcs	2025_07_26_AB_02_INF052_00_SEB.fcs
2025_07_26_AB_02_INF100_00_Ctrl.fcs	2025_07_26_AB_02_INF100_00_Ctrl.fcs
2025_07_26_AB_02_INF100_00_SEB.fcs	2025_07_26_AB_02_INF100_00_SEB.fcs
2025_07_26_AB_02_INF179_00_Ctrl.fcs	2025_07_26_AB_02_INF179_00_Ctrl.fcs
2025_07_26_AB_02_INF179_00_SEB.fcs	2025_07_26_AB_02_INF179_00_SEB.fcs

In the current form, the metadata is kinda empty, only containing rownames and the name column. However, we can notice that within the existing names, we have some useful information present between the underscores that we might be able to extract out and append as additional metadata columns.

Lets first off retrieve the metadata by assigning to its own object/variable:

```
Metadata <- pData(SFC_GatingSet)
str(Metadata)
```

```
‘data.frame’: 6 obs. of 1 variable: $ name: chr “2025_07_26_AB_02_INF052_00_Ctrl.fcs”
“2025_07_26_AB_02_INF052_00_SEB.fcs” “2025_07_26_AB_02_INF100_00_Ctrl.fcs”
“2025_07_26_AB_02_INF100_00_SEB.fcs” ...
```

As you can see, it is just a data.frame style object that we have encountered in the past. We can therefore use dplyr package functions to help us tidy the metadata to extract the additional information that we may need in the future. Let’s go ahead and load in dplyr

```
::: {.cell}
```

```
library(dplyr)
```

```
:::
```

```
::: {.cell}
```

```
library(stringr)
```

```
:::
```

And then create a new column for Condition so that we can better evaluate how the various extraction functions work, and also not risk messing up the underlying data while testing out how to use several new functions.

```
::: {.cell}
```

```
Testing <- Metadata |> mutate(Condition=name)
```

```
:::
```

Glancing at the current names, we can notice that if we wanted to extract the Condition (Ctrl, PPD, SEB), we would need to remove all character values up the final underscore (_), and before the “fcs”.

```
::: {.cell}
```

```
Testing |> pull(Condition)
```

```
:::
```

```
1_2025_07_26_AB_02_INF052_00_Ctrl.fcs 2_2025_07_26_AB_02_INF052_00_SEB.fcs  
3_2025_07_26_AB_02_INF100_00_Ctrl.fcs 4_2025_07_26_AB_02_INF100_00_SEB.fcs  
5_2025_07_26_AB_02_INF179_00_Ctrl.fcs 6_2025_07_26_AB_02_INF179_00_SEB.fcs
```

Within base R, two functions, `sub()` and `gsub()` are most frequently encountered when attempting to extract out a particular pattern. These rely on providing a regular expression (RegEx) pattern to match, and a replacement value to swap in its place (which can be left empty by providing a “”).

Consequently, if we were using `sub()`, our first step to replace everything up through the last underscore would be the RegEx expression “.*_”

```
sub(".*_", "", Testing$Condition)
```

```
[1] "Ctrl.fcs" "SEB.fcs" "Ctrl.fcs" "SEB.fcs" "Ctrl.fcs" [6] "SEB.fcs"
```

And we could follow up by matching the “fcs” pattern and leaving the replacement value as “ ” as well.

```
Intermediate <- sub(".*_", "", Testing$Condition)
sub(".fcs", "", Intermediate)
```

[1] "Ctrl" "SEB" "Ctrl" "SEB" "Ctrl" "SEB"

IMPORTANT NOTE: Please be advised, even with years of practice, I still will struggle to remember the proper RegEx syntax code to extract out a pattern from a long complicated character string. This is one use case where having a LLM provide a narrowly focused answer can be quite useful. Prompt example:

"I am working in R, using sub(), I want to keep everything after the last _ : 2025_07_26_AB_02_INF052_00_C

In addition to sub() and gsub() in base R, we can also achieve a similar outcome using the stringr package from the tidyverse, using the str_extract() function (after investigating the correct RegEx style syntax). To interpret this RegEx-inspired hieroglyphic, note that "^" is used for the start of a string, and "\$" for the end of the string. So we are extracting the pattern after the underscore "^_", but keeping the contents before the final .fcs

```
str_extract(Testing$Condition, "[^_]+(?=\\.fcs$)")
```

[1] "Ctrl" "SEB" "Ctrl" "SEB" "Ctrl" "SEB"

So, wrapping it all together, we could write the following to create a Condition column

```
Metadata |> mutate(Condition = name) |>
mutate(Condition=str_extract(Condition, "[^_]+(?=\\.fcs$)"))
```

	name
2025_07_26_AB_02_INF052_00_Ctrl.fcs	2025_07_26_AB_02_INF052_00_Ctrl.fcs
2025_07_26_AB_02_INF052_00_SEB.fcs	2025_07_26_AB_02_INF052_00_SEB.fcs
2025_07_26_AB_02_INF100_00_Ctrl.fcs	2025_07_26_AB_02_INF100_00_Ctrl.fcs
2025_07_26_AB_02_INF100_00_SEB.fcs	2025_07_26_AB_02_INF100_00_SEB.fcs
2025_07_26_AB_02_INF179_00_Ctrl.fcs	2025_07_26_AB_02_INF179_00_Ctrl.fcs
2025_07_26_AB_02_INF179_00_SEB.fcs	2025_07_26_AB_02_INF179_00_SEB.fcs
Condition	2025_07_26_AB_02_INF052_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF052_00_SEB.fcs
SEB	2025_07_26_AB_02_INF100_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF100_00_SEB.fcs
SEB	2025_07_26_AB_02_INF179_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF179_00_SEB.fcs
SEB	

Fortunately, extracting an actual pattern is a bit simpler, so we can similarly grab the specimen ID number.

```
TwoExtraColumns <- Metadata |> mutate(Condition = name) |>
  mutate(Condition=str_extract(Condition, "[^_]+(?:\\.fcs$)")) |>
  mutate(Specimen=str_extract(name, "INF\\d{3}")) |>
  select(Condition, Specimen)
TwoExtraColumns
```

Condition

```
2025_07_26_AB_02_INF052_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF052_00_SEB.fcs
SEB 2025_07_26_AB_02_INF100_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF100_00_SEB.fcs
SEB 2025_07_26_AB_02_INF179_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF179_00_SEB.fcs
SEB Specimen 2025_07_26_AB_02_INF052_00_Ctrl.fcs INF052 2025_07_26_AB_02_INF052_00_SEB.fcs
INF052 2025_07_26_AB_02_INF100_00_Ctrl.fcs INF100 2025_07_26_AB_02_INF100_00_SEB.fcs
INF100 2025_07_26_AB_02_INF179_00_Ctrl.fcs INF179 2025_07_26_AB_02_INF179_00_SEB.fcs
INF179
```

Updating Metadata

When binding columns from two separate data.frame objects in R, we can use the cbind() base R function.

```
UpdatedMetadata <- cbind(Metadata, TwoExtraColumns)
UpdatedMetadata
```

name

```
2025_07_26_AB_02_INF052_00_Ctrl.fcs    2025_07_26_AB_02_INF052_00_Ctrl.fcs
2025_07_26_AB_02_INF052_00_SEB.fcs    2025_07_26_AB_02_INF052_00_SEB.fcs
2025_07_26_AB_02_INF100_00_Ctrl.fcs    2025_07_26_AB_02_INF100_00_Ctrl.fcs
2025_07_26_AB_02_INF100_00_SEB.fcs    2025_07_26_AB_02_INF100_00_SEB.fcs
2025_07_26_AB_02_INF179_00_Ctrl.fcs    2025_07_26_AB_02_INF179_00_Ctrl.fcs
2025_07_26_AB_02_INF179_00_SEB.fcs    2025_07_26_AB_02_INF179_00_SEB.fcs
Condition 2025_07_26_AB_02_INF052_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF052_00_SEB.fcs
SEB 2025_07_26_AB_02_INF100_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF100_00_SEB.fcs
SEB 2025_07_26_AB_02_INF179_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF179_00_SEB.fcs
SEB Specimen 2025_07_26_AB_02_INF052_00_Ctrl.fcs INF052 2025_07_26_AB_02_INF052_00_SEB.fcs
INF052 2025_07_26_AB_02_INF100_00_Ctrl.fcs INF100 2025_07_26_AB_02_INF100_00_SEB.fcs
INF100 2025_07_26_AB_02_INF179_00_Ctrl.fcs INF179 2025_07_26_AB_02_INF179_00_SEB.fcs
INF179
```

Which we can then use to overwrite the existing metadata present for the GatingSet.

```
pData(SFC_GatingSet) <- UpdatedMetadata
pData(SFC_GatingSet)
```

```

                                name
2025_07_26_AB_02_INF052_00_Ctrl.fcs  2025_07_26_AB_02_INF052_00_Ctrl.fcs
2025_07_26_AB_02_INF052_00_SEB.fcs  2025_07_26_AB_02_INF052_00_SEB.fcs
2025_07_26_AB_02_INF100_00_Ctrl.fcs  2025_07_26_AB_02_INF100_00_Ctrl.fcs
2025_07_26_AB_02_INF100_00_SEB.fcs  2025_07_26_AB_02_INF100_00_SEB.fcs
2025_07_26_AB_02_INF179_00_Ctrl.fcs  2025_07_26_AB_02_INF179_00_Ctrl.fcs
2025_07_26_AB_02_INF179_00_SEB.fcs  2025_07_26_AB_02_INF179_00_SEB.fcs
Specimen 2025_07_26_AB_02_INF052_00_Ctrl.fcs INF052 2025_07_26_AB_02_INF052_00_SEB.fcs
INF052 2025_07_26_AB_02_INF100_00_Ctrl.fcs INF100 2025_07_26_AB_02_INF100_00_SEB.fcs
INF100 2025_07_26_AB_02_INF179_00_Ctrl.fcs INF179 2025_07_26_AB_02_INF179_00_SEB.fcs
INF179 Condition 2025_07_26_AB_02_INF052_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF052_00_SEB.fcs
SEB 2025_07_26_AB_02_INF100_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF100_00_SEB.fcs
SEB 2025_07_26_AB_02_INF179_00_Ctrl.fcs Ctrl 2025_07_26_AB_02_INF179_00_SEB.fcs
SEB
```

SFC

Having emerged from the metadata, let's return to focusing on our SFC dataset.

```
SFC_GatingSet
```

A GatingSet with 6 samples

colnames

When working with .fcs files that we have not worked with recently, it is often good to go ahead and check to see what fluorophores/markers are present at the start. This is also useful as different cytometer platforms have slightly different naming conventions. We can use `colnames()`

```
::: {.cell}
```

```
colnames(SFC_GatingSet)
```

...

[1] "Time" "SSC-W"
[3] "SSC-H" "SSC-A"
[5] "FSC-W" "FSC-H"
[7] "FSC-A" "SSC-B-W"
[9] "SSC-B-H" "SSC-B-A"
[11] "BUV395-A" "BUV563-A"
[13] "BUV615-A" "BUV661-A"
[15] "BUV737-A" "BUV805-A"
[17] "Pacific Blue-A" "BV480-A"
[19] "BV570-A" "BV605-A"
[21] "BV650-A" "BV711-A"
[23] "BV750-A" "BV786-A"
[25] "Alexa Fluor 488-A" "Spark Blue 550-A" [27] "Spark Blue 574-A" "RB613-A"
[29] "RB705-A" "RB780-A"
[31] "PE-A" "PE-Dazzle594-A"
[33] "PE-Cy5-A" "PE-Fire 700-A"
[35] "PE-Fire 744-A" "PE-Vio770-A"
[37] "APC-A" "Alexa Fluor 647-A" [39] "APC-R700-A" "Zombie NIR-A"
[41] "APC-Fire 750-A" "APC-Fire 810-A"
[43] "AF-A"

As we can see from the output, most of the fluorophores have an "-A" appended to the end, although the FSC and SSC parameters feature area, height and width ("-A", "-H", "-W") respectively.

markernames

If we wanted to retrieve the markers corresponding to the individual fluorophores, rather than attempting the breaking into the S4 object as we did during Week 03, we can take advantage of the `markernames()` function provided by the developers.

```
markernames(SFC_GatingSet)
```

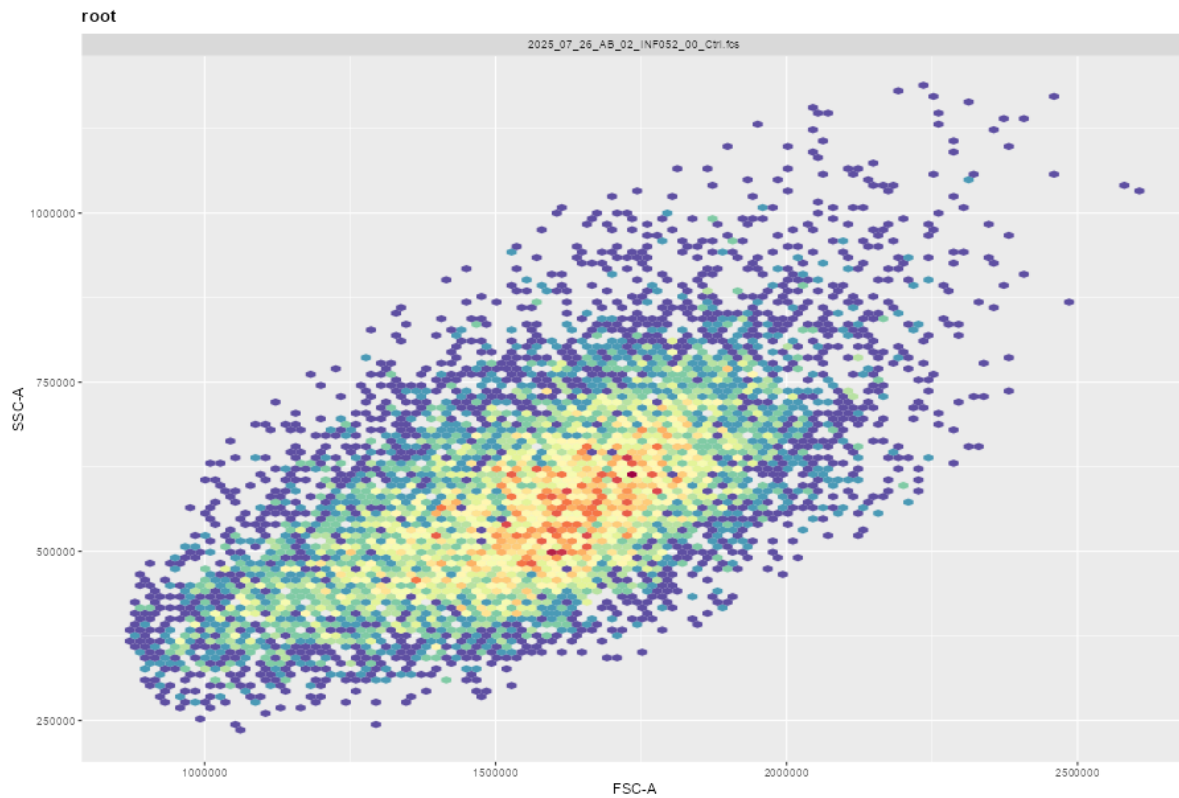
BUV395-A	BUV563-A
"CD62L"	"CD69"
BUV615-A	BUV661-A
"CCR4"	"Vd2"
BUV737-A	BUV805-A
"CD38"	"CD4"

Pacific Blue-A BV480-A “Dump” “CD161” BV570-A BV605-A “CD16” “CD45RA” BV650-A BV711-A “CD8” “Va7.2” BV750-A BV786-A “IFNg” “CCR6” Alexa Fluor 488-A Spark Blue 550-A “FoxP3” “CD3” Spark Blue 574-A RB613-A “CD45” “PD1” RB705-A RB780-A “CD26” “CXCR5” PE-A PE-Dazzle594-A “ICOS” “TNFa” PE-Cy5-A PE-Fire 700-A “CXCR3” “CD127” PE-Fire 744-A PE-Vio770-A “CD25” “HLA-DR” APC-A Alexa Fluor 647-A “CD39” “IL-2” APC-R700-A Zombie NIR-A “CD107a” “Viability” APC-Fire 750-A APC-Fire 810-A “CD27” “CCR7”

ggcyto

At this point, we can then use what we have learned about the ggcyto package to make sure that the FSC x SSC plotting is working correctly.

```
ggcyto(SFC_GatingSet[1], subset="root",
  aes(x="FSC-A", y="SSC-A")) + geom_hex(bins=100)
```



Please note, that for ggcyto it's just one set of [] after the GatingSet name to subset for plotting.

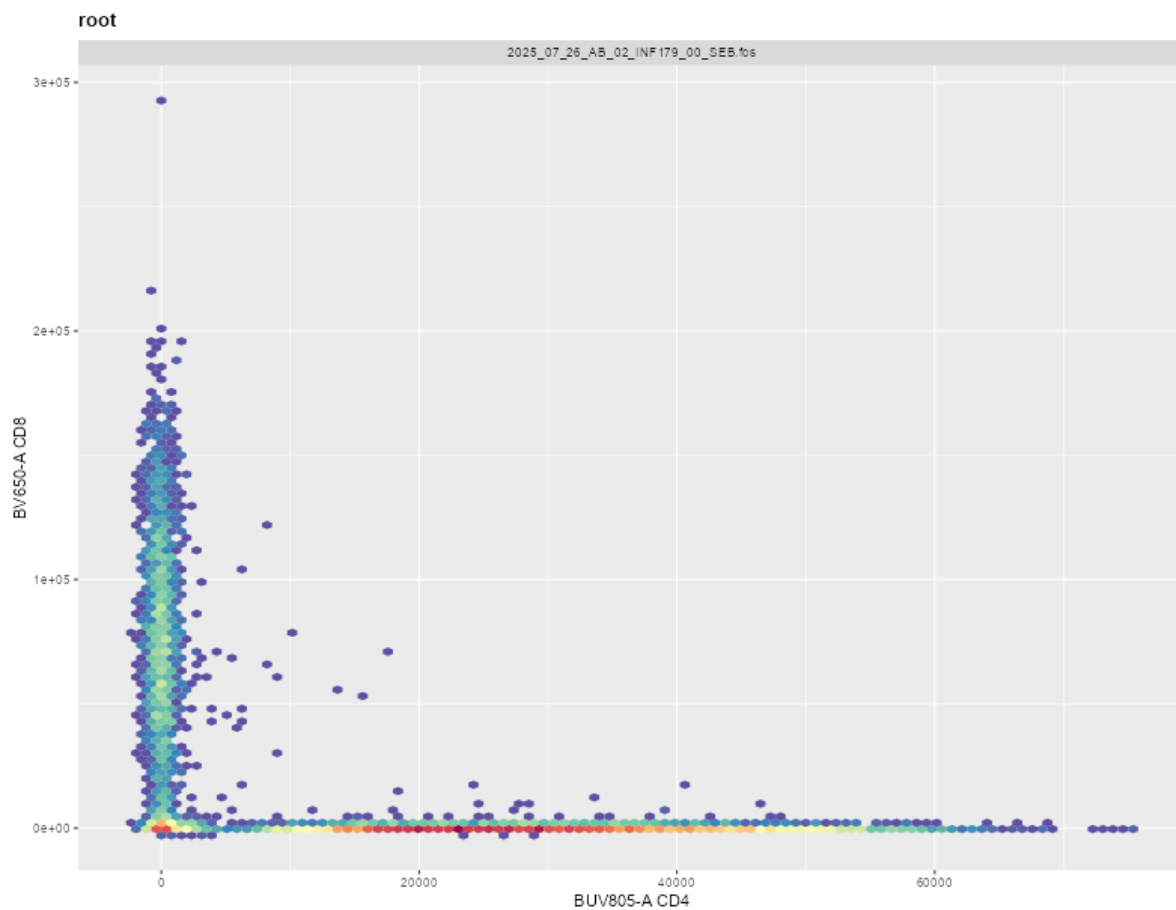

```
str(SFC_GatingSet[1])
```

Formal class 'GatingSet' [package "flowWorkspace"] with 2 slots ..@ pointer : ..@ transformation: list()

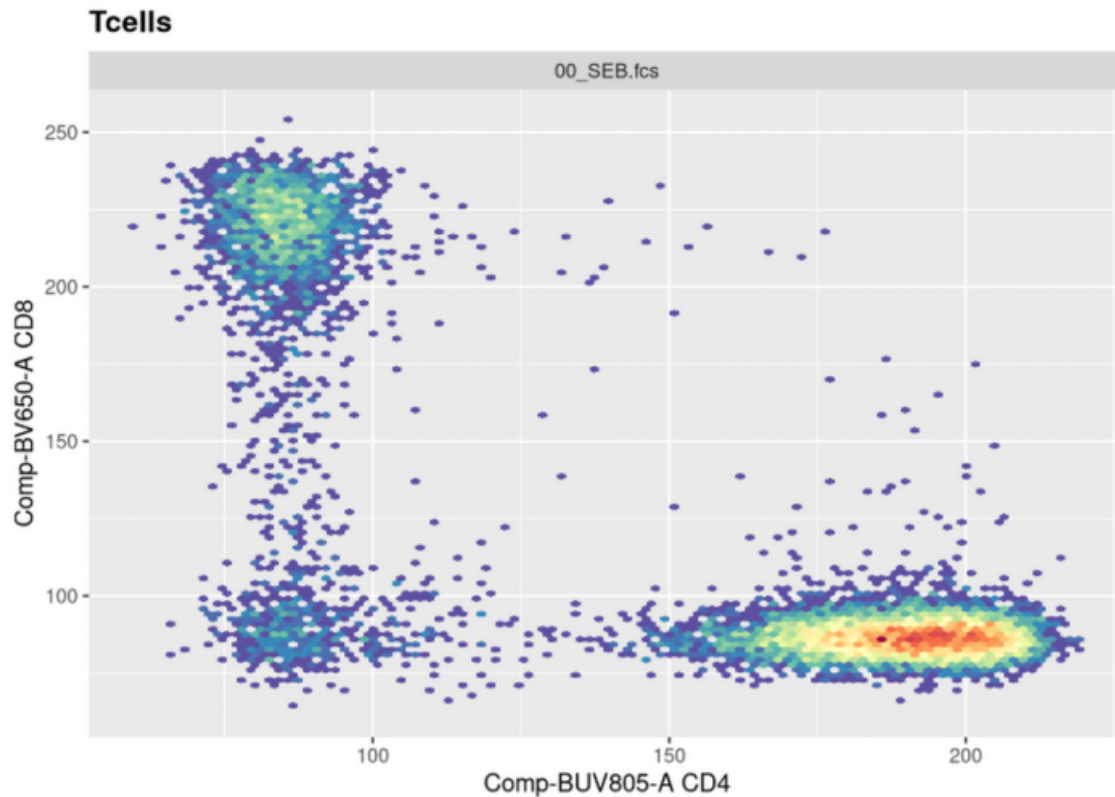
Untransformed Data

Let's go ahead and plot a couple of the fluorophores, to see what untransformed data in R looks like.

```
ggcyto(SFC_GatingSet[6], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```



As we can see, the scale currently appears linear, with a wide stretch of positive values for both CD4 and CD8 T cells. By contrast, you may remember the transformed version brought for this same .fcs file brought in from FlowJo via CytoML resembled the following



If we use the `gh_get_transformations` function, we can see that there are currently no transformations applied to the GatingSet.

```
gh_get_transformations(SFC_GatingSet[[1]])
```

```
list()
```

Transformers

`flowWorkspace` provides several default transforms (with the option to create your own). The functions in question return transformer objects, which are list that collect the required information for the transformations that are subsequently applied.

```
Logicle <- logicle_trans()
str(Logicle)
```

```
List of 9 $ name : chr "logicle" $ transform :function (x)
$ inverse :function (x)
```

```

$ d_transform : NULL $ d_inverse : NULL $ breaks :function (x)
$ minor_breaks:function (b, limits, n)
$ format :function (x)
$ domain : num [1:2] -Inf Inf - attr(*, "class")= chr "transform"
::: {.cell}

```

```

Biexponential <- flowjo_biexp_trans()
str(Biexponential)

```

```

:::
List of 9 $ name : chr "flowJo_biexp" $ transform :function (x, deriv = 0)
..- attr(, "type")= chr "biexp" ..- attr(, "parameters")=List of 5 .. ..$ channelRange: int 4096
.. ..$ maxValue : num 262144 .. ..$ neg : num 0 .. ..$ pos : num 4.5 .. ..$ widthBasis : num
-10 $ inverse :function (x, deriv = 0)
..- attr(, "type")= chr "biexp" ..- attr(, "parameters")=List of 5 .. ..$ channelRange: int 4096
.. ..$ maxValue : num 262144 .. ..$ neg : num 0 .. ..$ pos : num 4.5 .. ..$ widthBasis : num
-10 $ d_transform : NULL $ d_inverse : NULL $ breaks :function (x)
$ minor_breaks:function (b, limits, n)
$ format :function (x)
$ domain : num [1:2] -Inf Inf - attr(*, "class")= chr "transform"
::: {.cell}

```

```

Asinh <- flowjo_fasinh_trans()
str(Asinh)

```

```

:::
List of 9 $ name : chr "flowJo_fasinh" $ transform :function (x)
$ inverse :function (x)
$ d_transform : NULL $ d_inverse : NULL $ breaks :function (x)
$ minor_breaks:function (b, limits, n)
$ format :function (x)
$ domain : num [1:2] -Inf Inf - attr(*, "class")= chr "transform"
::: {.cell}

```

```

AsinhGML <- asinhtGml2_trans()
str(AsinhGML)

```

```

:::

```

```
List of 9 $ name : chr "asinhtGml2" $ transform :function (x)
$ inverse :function (x)
$ d_transform : NULL $ d_inverse : NULL $ breaks :function (x)
$ minor_breaks: function (b, limits, n)
$ format :function (x)
$ domain : num [1:2] -Inf Inf - attr(*, "class")= chr "transform"
```

As always, it can be worthwhile to first check the help documentation, to investigate the various arguments that can be used within the setup.

Column names to be transformed While having the transformation parameters is one component, the other is the fluorophores that are to be transformed. Recalling the `colnames()` function, we can see we have the following fluorophores for this panel.

```
::: {.cell}
```

```
SFC_Parameters <- colnames(SFC_GatingSet)
SFC_Parameters
```

```
:::
```

```
[1] "Time" "SSC-W"
[3] "SSC-H" "SSC-A"
[5] "FSC-W" "FSC-H"
[7] "FSC-A" "SSC-B-W"
[9] "SSC-B-H" "SSC-B-A"
[11] "BUV395-A" "BUV563-A"
[13] "BUV615-A" "BUV661-A"
[15] "BUV737-A" "BUV805-A"
[17] "Pacific Blue-A" "BV480-A"
[19] "BV570-A" "BV605-A"
[21] "BV650-A" "BV711-A"
[23] "BV750-A" "BV786-A"
[25] "Alexa Fluor 488-A" "Spark Blue 550-A" [27] "Spark Blue 574-A" "RB613-A"
[29] "RB705-A" "RB780-A"
[31] "PE-A" "PE-Dazzle594-A"
[33] "PE-Cy5-A" "PE-Fire 700-A"
[35] "PE-Fire 744-A" "PE-Vio770-A"
[37] "APC-A" "Alexa Fluor 647-A" [39] "APC-R700-A" "Zombie NIR-A"
[41] "APC-Fire 750-A" "APC-Fire 810-A"
[43] "AF-A"
```

We will only need to apply transformations to fluorophores, not to FSC, SSC or Time parameters. We will therefore need to remove these from the list. One way would be to use `[]` index

method, combining a `!` and `stringr::str_detect()` to remove for values that would be shared by those parameters

```
FluorophoresOnly <- SFC_Parameters[!stringr::str_detect
(SFC_Parameters, "FSC|SSC|Time")]
FluorophoresOnly
```

```
[1] "BUV395-A" "BUV563-A"
[3] "BUV615-A" "BUV661-A"
[5] "BUV737-A" "BUV805-A"
[7] "Pacific Blue-A" "BV480-A"
[9] "BV570-A" "BV605-A"
[11] "BV650-A" "BV711-A"
[13] "BV750-A" "BV786-A"
[15] "Alexa Fluor 488-A" "Spark Blue 550-A" [17] "Spark Blue 574-A" "RB613-A"
[19] "RB705-A" "RB780-A"
[21] "PE-A" "PE-Dazzle594-A"
[23] "PE-Cy5-A" "PE-Fire 700-A"
[25] "PE-Fire 744-A" "PE-Vio770-A"
[27] "APC-A" "Alexa Fluor 647-A" [29] "APC-R700-A" "Zombie NIR-A"
[31] "APC-Fire 750-A" "APC-Fire 810-A"
[33] "AF-A"
```

transformerList

Now that we have the fluorophore columns identified, using the `transformerList()` function we can combine them with the `Transformer` object we previously created.

```
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
MyBiexTransform
```

```
$BUV395-A Transformer: flowJo_biexp [-Inf, Inf]
$BUV563-A Transformer: flowJo_biexp [-Inf, Inf]
$BUV615-A Transformer: flowJo_biexp [-Inf, Inf]
$BUV661-A Transformer: flowJo_biexp [-Inf, Inf]
$BUV737-A Transformer: flowJo_biexp [-Inf, Inf]
$BUV805-A Transformer: flowJo_biexp [-Inf, Inf]
$Pacific Blue-A Transformer: flowJo_biexp [-Inf, Inf]
```

\$BV480-A Transformer: flowJo_biexp [-Inf, Inf]
 \$BV570-A Transformer: flowJo_biexp [-Inf, Inf]
 \$BV605-A Transformer: flowJo_biexp [-Inf, Inf]
 \$BV650-A Transformer: flowJo_biexp [-Inf, Inf]
 \$BV711-A Transformer: flowJo_biexp [-Inf, Inf]
 \$BV750-A Transformer: flowJo_biexp [-Inf, Inf]
 \$BV786-A Transformer: flowJo_biexp [-Inf, Inf]
 \$Alexa Fluor 488-A Transformer: flowJo_biexp [-Inf, Inf]
 \$Spark Blue 550-A Transformer: flowJo_biexp [-Inf, Inf]
 \$Spark Blue 574-A Transformer: flowJo_biexp [-Inf, Inf]
 \$RB613-A Transformer: flowJo_biexp [-Inf, Inf]
 \$RB705-A Transformer: flowJo_biexp [-Inf, Inf]
 \$RB780-A Transformer: flowJo_biexp [-Inf, Inf]
 \$PE-A Transformer: flowJo_biexp [-Inf, Inf]
 \$PE-Dazzle594-A Transformer: flowJo_biexp [-Inf, Inf]
 \$PE-Cy5-A Transformer: flowJo_biexp [-Inf, Inf]
 \$PE-Fire 700-A Transformer: flowJo_biexp [-Inf, Inf]
 \$PE-Fire 744-A Transformer: flowJo_biexp [-Inf, Inf]
 \$PE-Vio770-A Transformer: flowJo_biexp [-Inf, Inf]
 \$APC-A Transformer: flowJo_biexp [-Inf, Inf]
 \$Alexa Fluor 647-A Transformer: flowJo_biexp [-Inf, Inf]
 \$APC-R700-A Transformer: flowJo_biexp [-Inf, Inf]
 \$Zombie NIR-A Transformer: flowJo_biexp [-Inf, Inf]
 \$APC-Fire 750-A Transformer: flowJo_biexp [-Inf, Inf]
 \$APC-Fire 810-A Transformer: flowJo_biexp [-Inf, Inf]
 \$AF-A Transformer: flowJo_biexp [-Inf, Inf]
 attr(,"class") [1] "transformerList" "list"

```
str(MyBiexTransform[1:3])
```

```
List of 3 $ BUV395-A:List of 9 ..$ name : chr "flowJo_biexp" ..$ transform :function (x, deriv
= 0)
.. .. attr(, "type")= chr "biexp" .. .. attr(, "parameters")=List of 5 .. .. ..$ channelRange:
int 4096 .. .. ..$ maxValue : num 262144 .. .. ..$ neg : num 0 .. .. ..$ pos : num 4.5 .. .. ..$
widthBasis : num -10 ..$ inverse :function (x, deriv = 0)
.. .. attr(, "type")= chr "biexp" .. .. attr(, "parameters")=List of 5 .. .. ..$ channelRange:
int 4096 .. .. ..$ maxValue : num 262144 .. .. ..$ neg : num 0 .. .. ..$ pos : num 4.5 .. .. ..$
widthBasis : num -10 ..$ d_transform : NULL ..$ d_inverse : NULL ..$ breaks :function (x)
..$ minor_breaks:function (b, limits, n)
..$ format :function (x)
..$ domain : num [1:2] -Inf Inf .. attr(, "class")= chr "transform" $ BUV563-A:List of 9 ..$
name : chr "flowJo_biexp" ..$ transform :function (x, deriv = 0)
.. .. attr(, "type")= chr "biexp" .. .. attr(, "parameters")=List of 5 .. .. ..$ channelRange:
int 4096 .. .. ..$ maxValue : num 262144 .. .. ..$ neg : num 0 .. .. ..$ pos : num 4.5 .. .. ..$
widthBasis : num -10 ..$ inverse :function (x, deriv = 0)
.. .. attr(, "type")= chr "biexp" .. .. attr(, "parameters")=List of 5 .. .. ..$ channelRange:
int 4096 .. .. ..$ maxValue : num 262144 .. .. ..$ neg : num 0 .. .. ..$ pos : num 4.5 .. .. ..$
widthBasis : num -10 ..$ d_transform : NULL ..$ d_inverse : NULL ..$ breaks :function (x)
..$ minor_breaks:function (b, limits, n)
..$ format :function (x)
..$ domain : num [1:2] -Inf Inf .. attr(, "class")= chr "transform" $ BUV615-A:List of 9 ..$
name : chr "flowJo_biexp" ..$ transform :function (x, deriv = 0)
.. .. attr(, "type")= chr "biexp" .. .. attr(, "parameters")=List of 5 .. .. ..$ channelRange:
int 4096 .. .. ..$ maxValue : num 262144 .. .. ..$ neg : num 0 .. .. ..$ pos : num 4.5 .. .. ..$
widthBasis : num -10 ..$ inverse :function (x, deriv = 0)
.. .. attr(, "type")= chr "biexp" .. .. attr(, "parameters")=List of 5 .. .. ..$ channelRange:
int 4096 .. .. ..$ maxValue : num 262144 .. .. ..$ neg : num 0 .. .. ..$ pos : num 4.5 .. .. ..$
widthBasis : num -10 ..$ d_transform : NULL ..$ d_inverse : NULL ..$ breaks :function (x)
..$ minor_breaks:function (b, limits, n)
..$ format :function (x)
..$ domain : num [1:2] -Inf Inf .. attr(*, "class")= chr "transform"
```

```
str(MyBiexTransform$`APC-A`)
```

```
List of 9 $ name : chr "flowJo_biexp" $ transform :function (x, deriv = 0)
.. attr(, "type")= chr "biexp" .. attr(, "parameters")=List of 5 .. ..$ channelRange: int 4096
.. ..$ maxValue : num 262144 .. ..$ neg : num 0 .. ..$ pos : num 4.5 .. ..$ widthBasis : num
-10 $ inverse :function (x, deriv = 0)
.. attr(, "type")= chr "biexp" .. attr(, "parameters")=List of 5 .. ..$ channelRange: int 4096
```

```
.. ..$ maxValue : num 262144 .. ..$ neg : num 0 .. ..$ pos : num 4.5 .. ..$ widthBasis : num
-10 $ d_transform : NULL $ d_inverse : NULL $ breaks :function (x)
$ minor_breaks: function (b, limits, n)
$ format :function (x)
$ domain : num [1:2] -Inf Inf - attr(*, "class")= chr "transform"
```

As you can notice, for each of the fluorophores we specified, the transformer object with the desired parameters has now been added as its own list entry. The final step is to then apply it to the GatingSet.

```
::: {.cell}
```

```
transform(SFC_GatingSet, MyBiexTransform)
```

```
:::
```

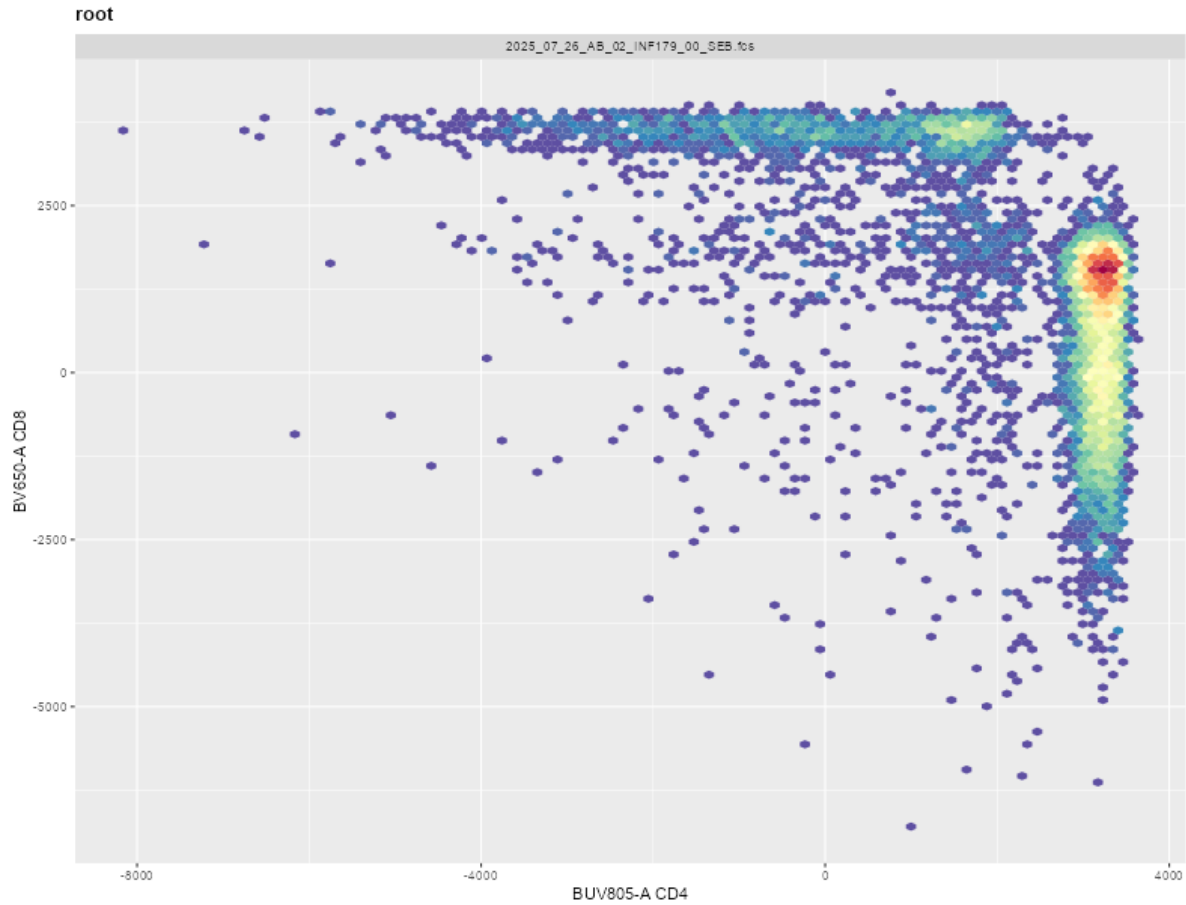
A GatingSet with 6 samples

If we now replot without having provided any arguments, we get the following:

```
::: {.cell}
```

```
ggcyto(SFC_GatingSet[6], subset="root",
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

```
:::
```

As we can see, applying a default transformation to a random .fcs file is not quite the way to go. We will need to look up some of the specific parameter arguments that need to be provided. Going back to our transformer function:

```
::: {.cell}
```

```
?flowjo_biexp_trans
```

```
:::
```

to see the help documentation. In this case, the help documentation doesn't provide much immediately. Part of this is that `flowjo_biexp_trans()` appears to be a wrapper function, which passes arguments on to the `flowJoTrans()` function. If we look this one up:

```
::: {.cell}
```

```
?flowJoTrans
```

```
:::
```

Under usage, we can see what the default options and arguments are.

Transformation Arguments Lets evaluate what effect changing each of these arguments in has on visualizing our .fcs file. Our current visual is:

```
::: {.cell}
```

```
SFC_cytoset <- load_cytoset_from_fcs(SFC_files,  
trunc_max_range = FALSE, transformation = FALSE)  
SFC_GatingSet <- GatingSet(SFC_cytoset)  
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,  
pos=4.5, neg=0, widthBasis=-10)  
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)  
transform(SFC_GatingSet, MyBiexTransform)
```

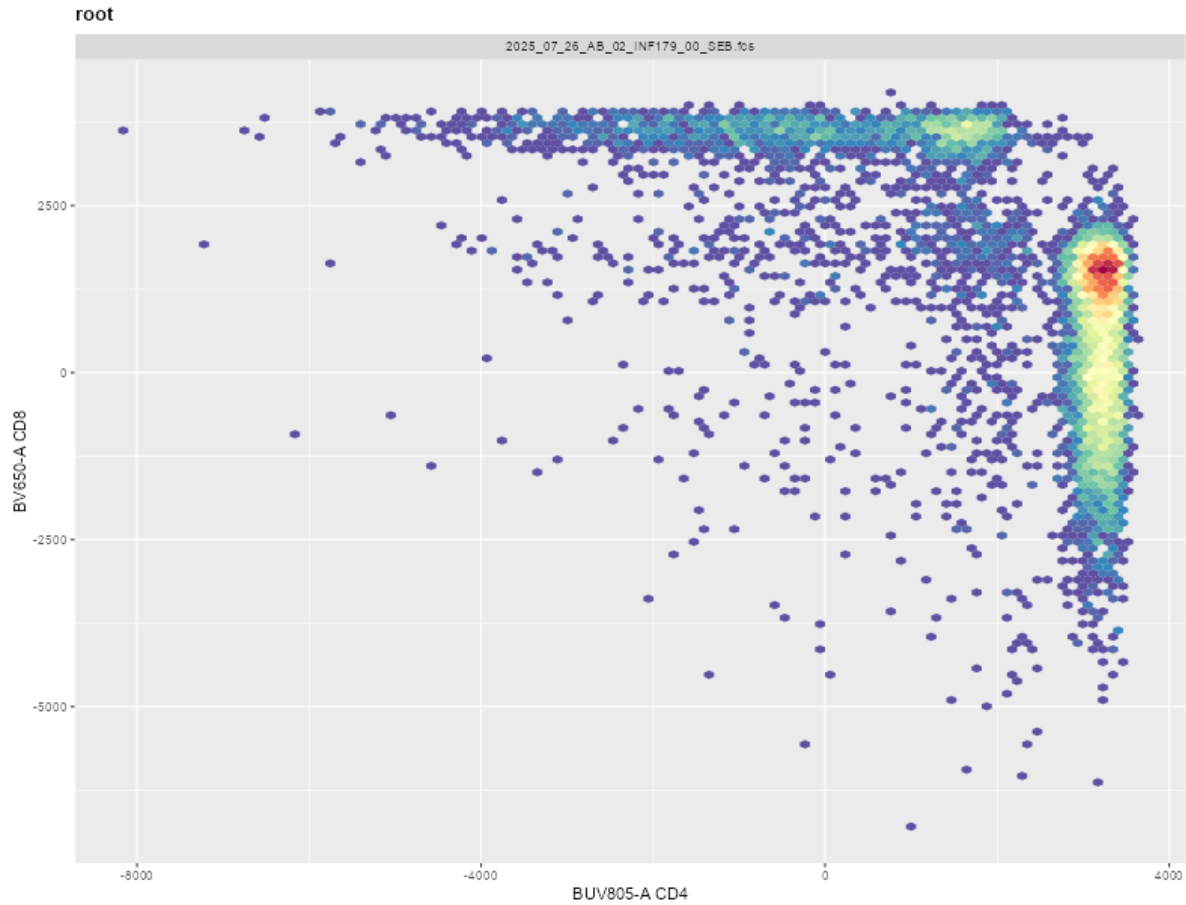
```
:::
```

A GatingSet with 6 samples

```
::: {.cell}
```

```
ggcyto(SFC_GatingSet[6], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

```
:::
```



We will also need a way to validate whether they are being updated in the background. We can revisit the output of `gh_get_transformations()` function from earlier, and specify just the first specimen, and the BUV805 fluorophore:

```
::: {.cell}
```

```
gh_get_transformations(SFC_GatingSet[[1]])$`BUV805-A`
```

```
:::
```

(a serial of codes that I erased because of problems rendering with latex)

```
<bytecode: 0x000001f3ea1dbe20> <environment: 0x000001f3ed0237e0> attr(,"type") [1] "bi-  
exp" attr(,"parameters") attr(,"parameters")$channelRange [1] 4096
```

```
attr(,"parameters")$maxValue [1] 262144
```

```
attr(,"parameters")$neg [1] 0
```

```
attr(,"parameters")$pos [1] 4.5  
attr(,"parameters")$widthBasis [1] -10
```

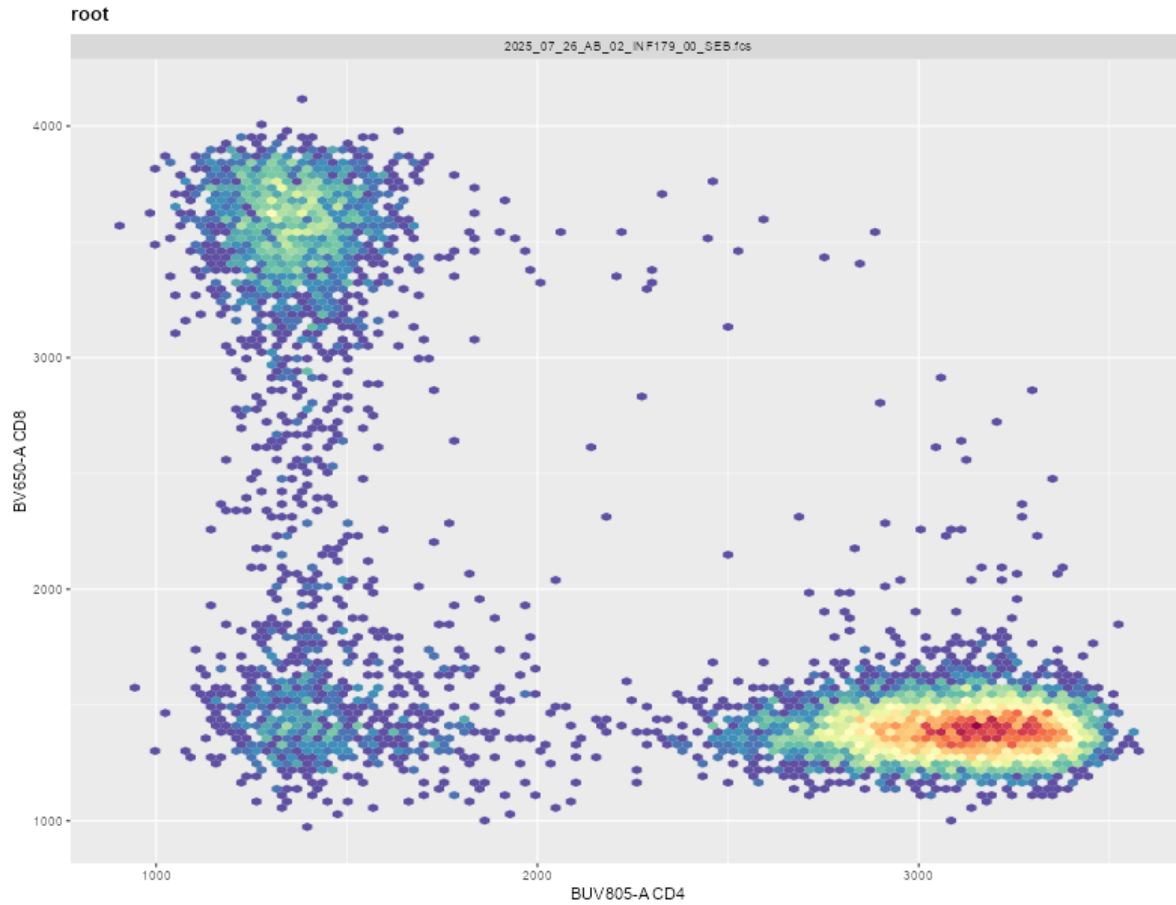
width

Before we go messing with any of the other setting, let's tackle width, which tends to be the one that gets missed by the default most often. Normally, in FlowJo most of my plots are set with biexponential transform width of around -1000, so lets go ahead and switch in that value.

```
SFC_cytoset <- load_cytoset_from_fcs(SFC_files,  
  truncate_max_range = FALSE, transformation = FALSE)  
SFC_GatingSet <- GatingSet(SFC_cytoset)  
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,  
  pos=4.5, neg=0, widthBasis=-1000)  
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)  
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 6 samples

```
ggcyto(SFC_GatingSet[6], subset="root",  
  aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```



Boom! That is more in line with what we were looking for. Let's still check a couple other values.

```
::: {.cell}
```

```
SFC_cytoaset <- load_cytoaset_from_fcs(SFC_files,
truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoaset)
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
pos=4.5, neg=0, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

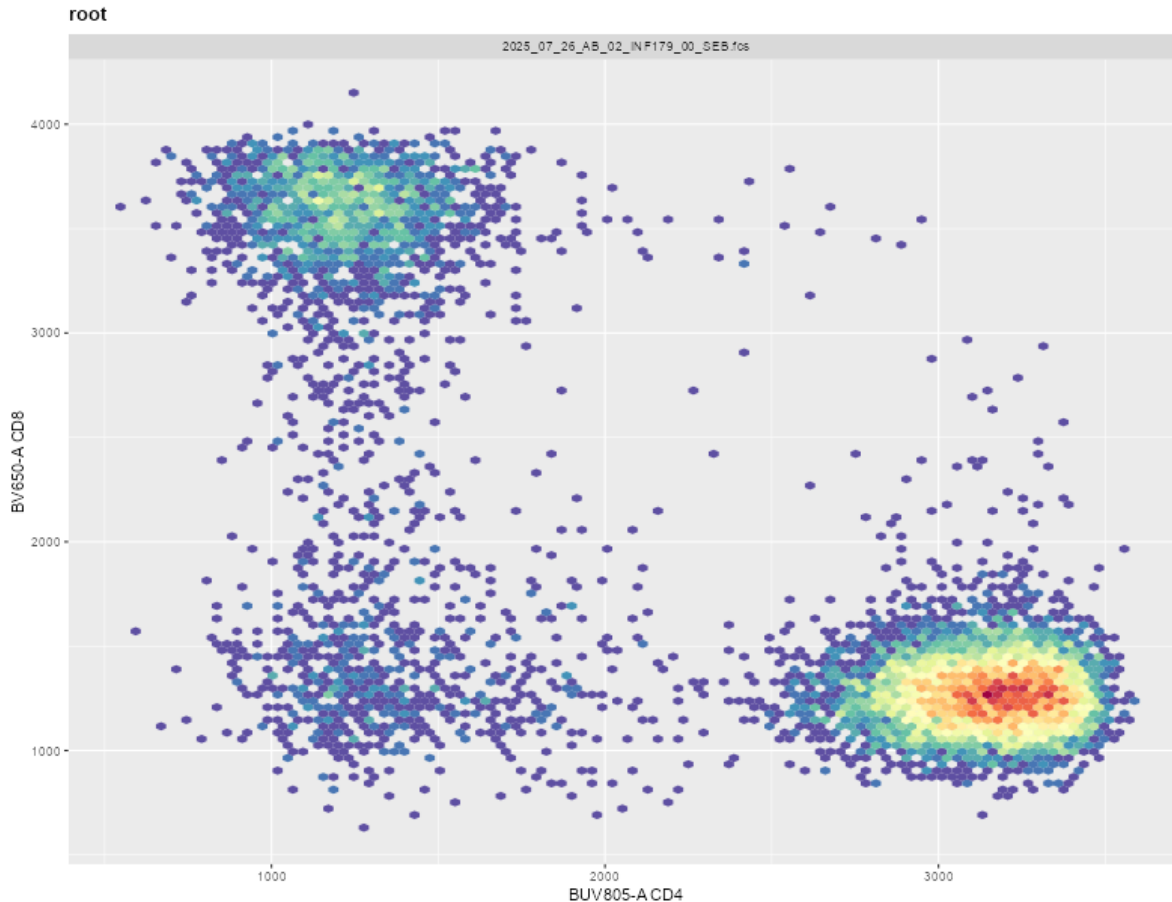
```
:::
```

A GatingSet with 6 samples

```
::: {.cell}
```

```
ggcyto(SFC_GatingSet[6], subset="root",
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

```
:::
```



A width value of -500 still has the negative population relative condensed, with the CD4+ cells into a more circular format. Let's try -100 next

```
::: {.cell}
```

```
SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,
truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoSet)
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
pos=4.5, neg=0, widthBasis=-100)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

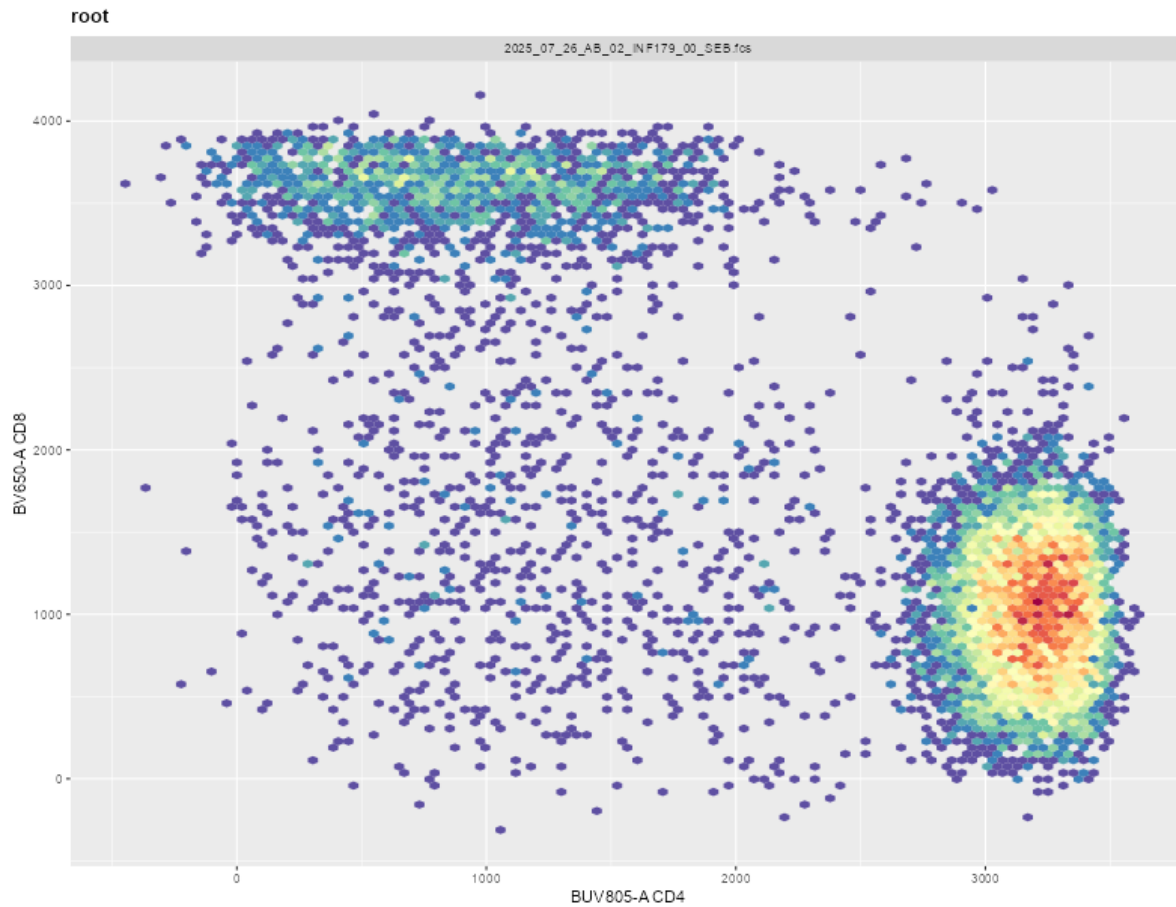
:::

A GatingSet with 6 samples

::: {.cell}

```
ggcyto(SFC_GatingSet[6], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

:::



And nope, too far. Let's keep -500 for now.

channelRange Now that width is set closer to what we would expect, lets alter the next couple arguments and see if they have any effect. The argument channelRange is currently set for 4096. Lets double it:

::: {.cell}

```

SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,
trunc_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoSet)
Biexponential <- flowjo_biexp_trans(channelRange=8192, maxValue=262144,
pos=4.5, neg=0, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)

```

:::

A GatingSet with 6 samples

::: {.cell}

```

ggcyto(SFC_GatingSet[6], subset="root",
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)

```

:::



```
::: {.cell}
```

```
gh_get_transformations(SFC_GatingSet[[1]])$`BV805-A`
```

```
:::
```

(a serial of codes that I erased because of problems rendering with latex)

```
<bytecode: 0x0000021b85a6b6f0> <environment: 0x0000021bf5f7ff40> attr(,"type") [1] "bi-  
exp" attr(,"parameters") attr(,"parameters")$channelRange [1] 8192
```

```
attr(,"parameters")$maxValue [1] 262144
```

```
attr(,"parameters")$neg [1] 0
```

```
attr(,"parameters")$pos [1] 4.5
```

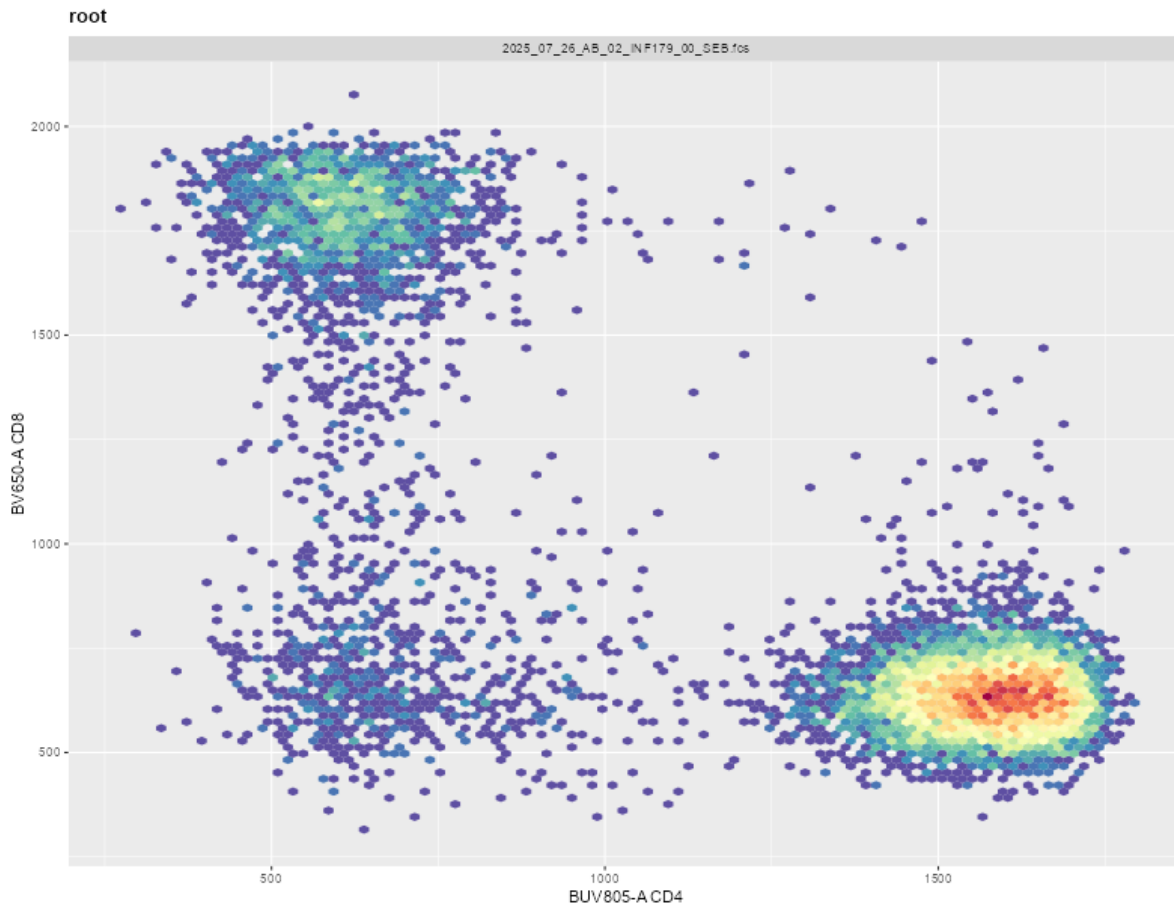
```
attr(,"parameters")$widthBasis [1] -500
```

Not much apparent change, so lets go the opposite direction and reduce the original value by half.

```
SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,  
truncate_max_range = FALSE, transformation = FALSE)  
SFC_GatingSet <- GatingSet(SFC_cytoSet)  
Biexponential <- flowjo_biexp_trans(channelRange=2048, maxValue=262144,  
pos=4.5, neg=0, widthBasis=-500)  
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)  
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 6 samples

```
ggcyto(SFC_GatingSet[6], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```



```
gh_get_transformations(SFC_GatingSet[[1]])$`BUV805-A`
```

(a serial of codes that I erased because of problems rendering with latex)

```
<bytecode: 0x0000021b85a6b6f0> <environment: 0x0000021b8be63d18> attr(,"type") [1]  
"biexp" attr(,"parameters") attr(,"parameters")$channelRange [1] 2048  
attr(,"parameters")$maxValue [1] 262144  
attr(,"parameters")$neg [1] 0  
attr(,"parameters")$pos [1] 4.5  
attr(,"parameters")$widthBasis [1] -500
```

Few minor changes, but not much otherwise. So lets proceed on to the next argument!

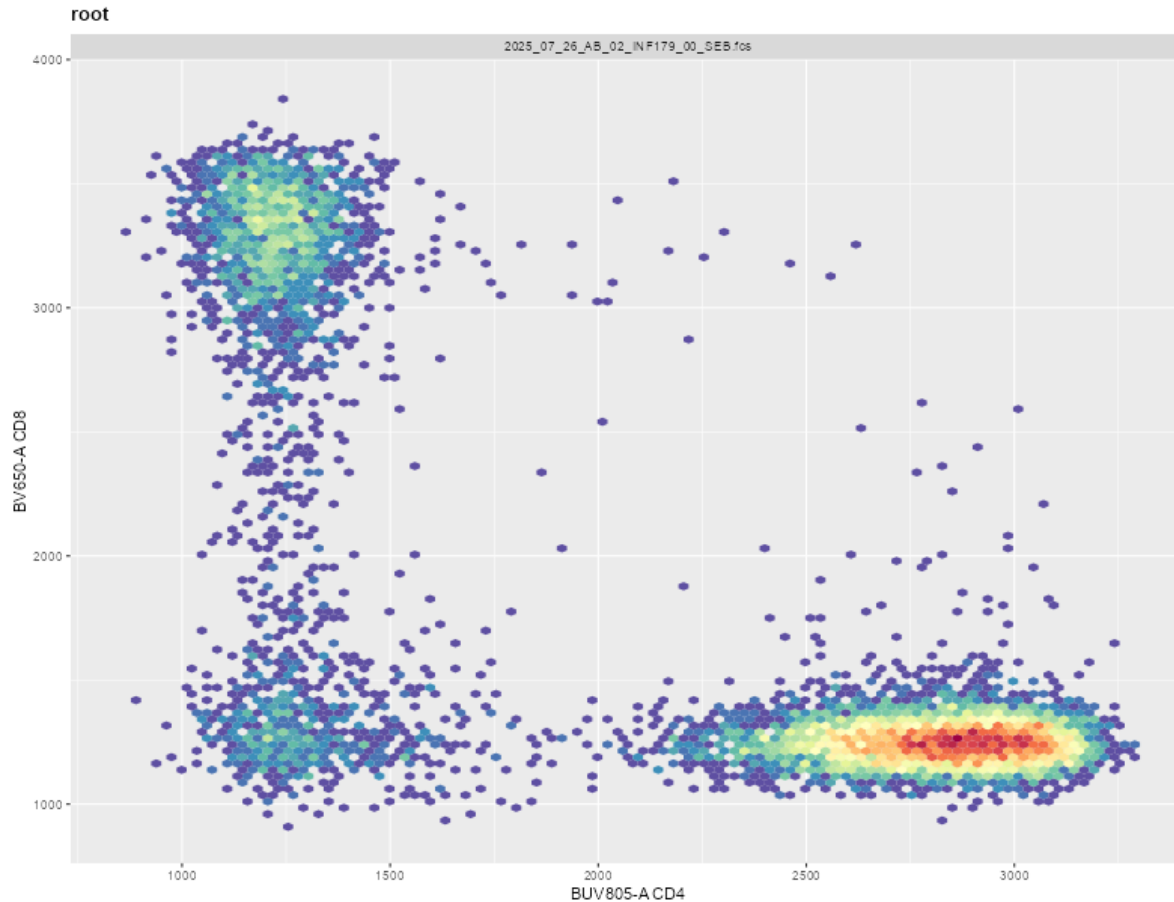
maxValue

Same procedure, the current maxValue is 262144, lets double it and see what happens!

```
SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,  
trunc_max_range = FALSE, transformation = FALSE)  
SFC_GatingSet <- GatingSet(SFC_cytoSet)  
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=524288,  
pos=4.5, neg=0, widthBasis=-500)  
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)  
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 6 samples

```
ggcyto(SFC_GatingSet[6], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```



You will notice, that when we doubled the maxValue, we are phenocopying the result we got when we had width set to -1000, with the CD4 population. This is worth noting. Lets now try instead to reduce it!

```
SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,
  truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoSet)
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=131072,
  pos=4.5, neg=0, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 6 samples

```
ggcyto(SFC_GatingSet[6], subset="root",
  aes(x="BV805-A", y="BV650-A")) + geom_hex(bins=100)
```



And likewise, we are going the opposite to our intended effect. As `maxValue` can in part be determined by our instrument settings (as well as individual fluorophores), this is an argument we should keep an eye on. Let's leave it at its current default for now.

pos

The next argument `pos` corresponds to the number of positive decades (this expression should be wrong... or not, but I don't understand). It is currently set at 4.5. Although this can be increased on some instrument settings. Let's walk it up to 5!

```
::: {.cell}
```

```
SFC_cytoset <- load_cytoset_from_fcs(SFC_files,
truncat_max_range = FALSE, transformation = FALSE)
SFC_gatingSet <- GatingSet(SFC_cytoset)
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
```

```
pos=5, neg=0, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

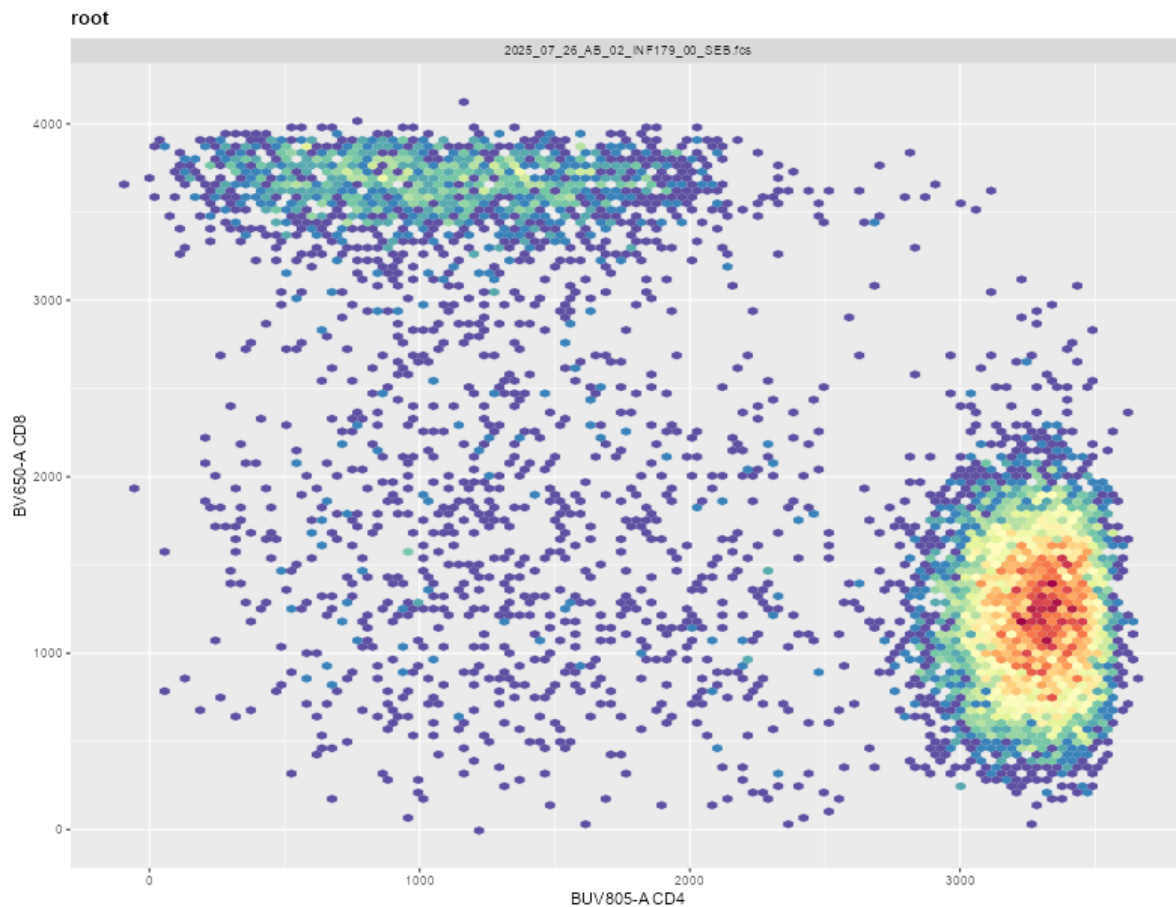
:::

A GatingSet with 6 samples

::: {.cell}

```
ggcyto(SFC_GatingSet[6], subset="root",
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

:::



And on first glance no. If we had our settings for 5 positive decades, we would need to adjust both the maxVal and widthBasis to return to a closer to expected shape.

```

SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,
trunc_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoSet)
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=524288,
pos=5, neg=0, widthBasis=-1000)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)

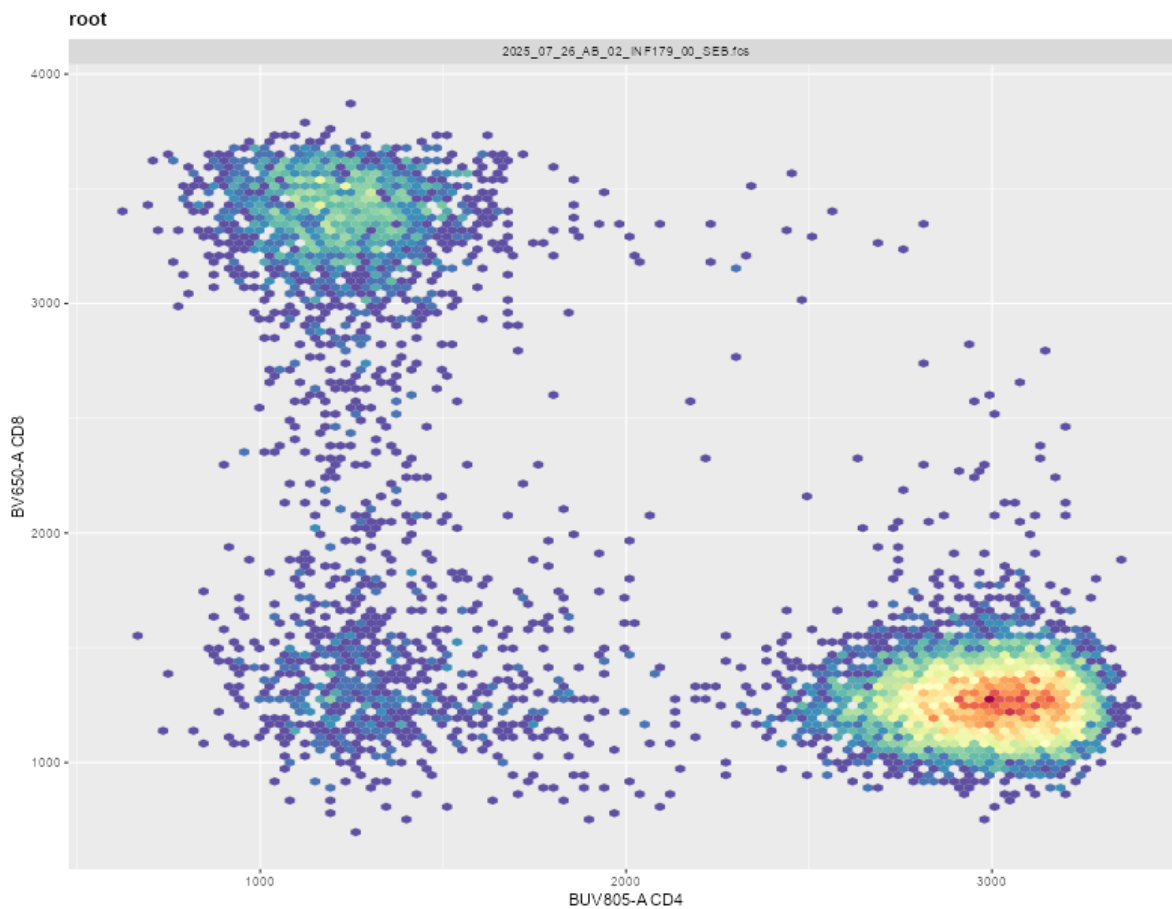
```

A GatingSet with 6 samples

```

ggcyto(SFC_GatingSet[6], subset="root",
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)

```



neg We have a similar story for adding negative decades. Adding an extra negative decade doesn't seem to noticeably affect these two fluorophores.

```
::: {.cell}
```

```
SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,  
trunc_max_range = FALSE, transformation = FALSE)  
SFC_GatingSet <- GatingSet(SFC_cytoSet)  
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,  
pos=4.5, neg=1, widthBasis=-500)  
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)  
transform(SFC_GatingSet, MyBiexTransform)
```

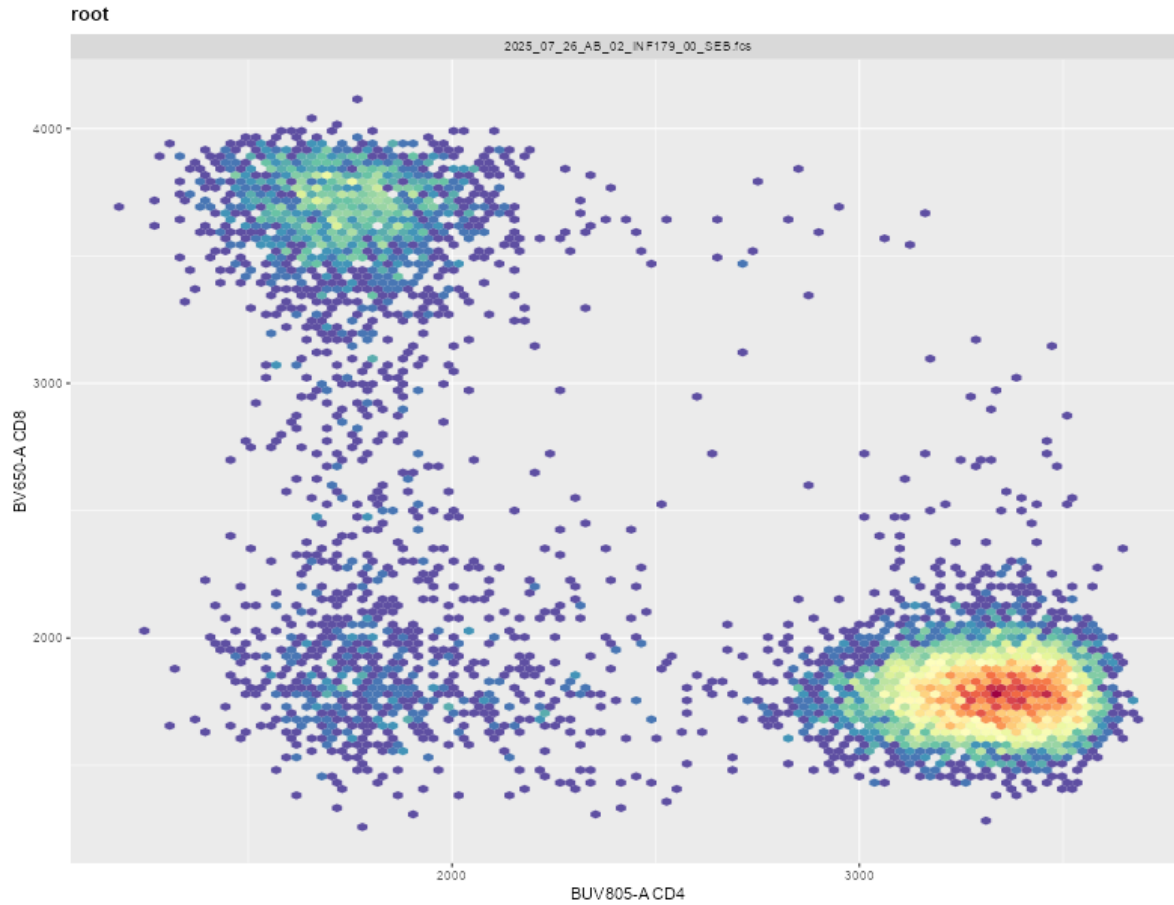
```
:::
```

A GatingSet with 6 samples

```
::: {.cell}
```

```
ggcyto(SFC_GatingSet[6], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

```
:::
```

It is only when we increase to 2 (i.e. -2 decades) that we start to see a spreading effect.

```
::: {.cell}
```

```
SFC_cytoSet <- load_cytoSet_from_fcs(SFC_files,
  truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoSet)
Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
  pos=4.5, neg=2, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

```
:::
```

A GatingSet with 6 samples

```
::: {.cell}
```

```
ggcyto(SFC_GatingSet[6], subset="root",
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```

...



Recap: summary We have looked at how the various arguments can affect the overall visualization, and seen how some of these are interconnected. It might be worthwhile to verify what your own instrument settings are at first (especially in context of maxVal and positive decads), and then use these when starting in R for the first time.

MC

Having now had some experience applying transformations to SFC data, let's turn and look at how to apply arsinh to our mass cytometry (CyTOF) dataset. Lets once again make sure we have the correct .fcs files

MC_files

1_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/G1_2_concatenated.txt_m039A.1147901.fcs 2_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/G1_2_concatenated.txt_m039B.1147889.fcs 3_C:/Users/Macarena/CytometryInR/course/07_Transformations/data/G1_2_concatenated.txt_m039C.1147913.fcs

And modify the steps from Week 05 to load them into a GatingSet object.

```
MC_cytoset <- load_cytoset_from_fcs(MC_files, truncate_max_range = FALSE,
transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoset)

MC_GatingSet
```

A GatingSet with 3 samples

Let's examine the names for the metal tags

```
colnames(MC_GatingSet)
```

```
[1] "Ce140Di" "Ce142Di"
[3] "Dy161Di" "Dy162Di"
[5] "Dy163Di" "Dy164Di"
[7] "Er166Di" "Er167Di"
[9] "Er168Di" "Er170Di"
[11] "Eu151Di" "Eu153Di"
[13] "Event_length" "Gd155Di"
[15] "Gd156Di" "Gd158Di"
[17] "Gd160Di" "Ho165Di"
[19] "Ir191Di" "Ir193Di"
[21] "Lu175Di" "Lu176Di"
[23] "Nd142Di" "Nd143Di"
[25] "Nd144Di" "Nd145Di"
[27] "Nd146Di" "Nd148Di"
[29] "Nd150Di" "Pd102Di"
[31] "Pd104Di" "Pd105Di"
[33] "Pd106Di" "Pd108Di"
[35] "Pd110Di" "Pr141Di"
[37] "Pt195Di" "Sm147Di"
[39] "Sm149Di" "Sm152Di"
[41] "Sm154Di" "Tb159Di"
```

```
[43] "Tm169Di" "Y89Di"
[45] "Yb171Di" "Yb172Di"
[47] "Yb173Di" "Yb174Di"
[49] "Yb176Di" "Time"
```

And the corresponding markers associated with each

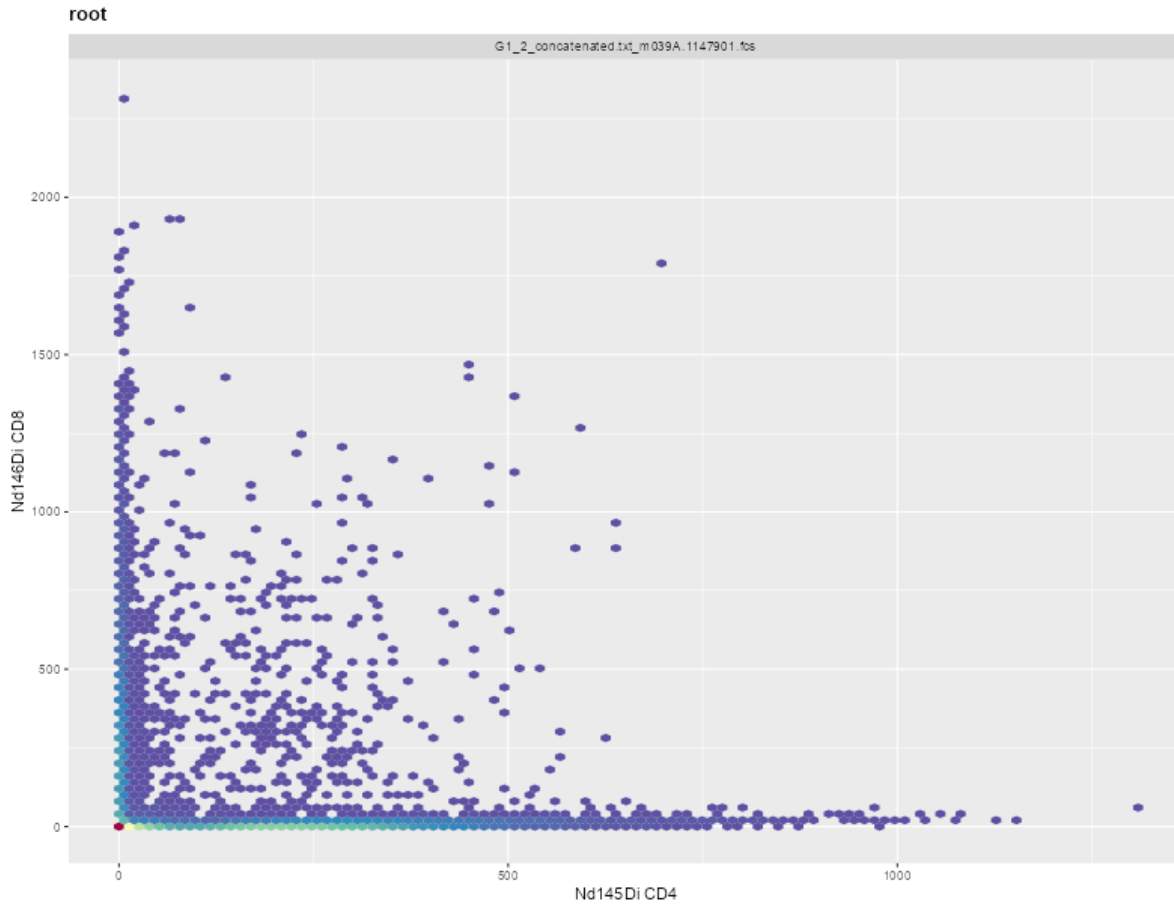
```
markernames(MC_GatingSet)
```

```
Ce140Di
"Beads"
Ce142Di
"Beads"
Dy161Di
```

```
"H2BK120Ub-macroH2A" Dy162Di "CrotonylK-H3K4me3" Dy163Di "H3R2cit-H2A.Z"
Dy164Di "H3K14ac-H3K36me1" Er166Di "CD33" Er167Di "CD16" Er168Di "H4K16ac-
H4K20me1" Er170Di "CD3" Eu151Di "H3K23ac-H3K27me1" Eu153Di "H2BS14ph-
H3K36me2" Gd155Di "CD11c" Gd156Di "H3K18ac-H4K20me2" Gd158Di "H3K56ac-H3.3"
Gd160Di "PADI4-H4K20me3" Ho165Di "H33Xcit-H3K27me3" Ir191Di "DNA" Ir193Di
"DNA" Lu175Di "CD19" Lu176Di "beads" Nd142Di "gammaH2AX-Rme1" Nd143Di
"H2BK5ac-Rme2sym" Nd144Di "H3S10ph-H3K4me2" Nd145Di "CD4" Nd146Di "CD8"
Nd148Di "CD34" Nd150Di "H3.3S31ph-H3K36me3" Pd102Di "Barcode" Pd104Di "Bar-
code" Pd105Di "Barcode" Pd106Di "Barcode" Pd108Di "Barcode" Pd110Di "Barcode"
Pr141Di "H3" Pt195Di "livedead" Sm147Di "H4K5ac-H3K9me2" Sm149Di "CleavedH3T22-
H3K9me1" Sm152Di "H3K9ac-Rme2asy" Sm154Di "H2AK119Ub-H3K27me2" Tb159Di
"CD14" Tm169Di "CD123" Y89Di "CD45" Yb171Di "CD38" Yb172Di "CD56" Yb173Di
"H4" Yb174Di "H3K27ac-CENPA" Yb176Di "HLADR"
```

And lets go ahead and visualize the CD4 x CD8 plot for the first specimen.

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```



We can then select the particular markers we are interested, that we want to end up transforming. For this example, let's say we wanted to remove the multiple Barcode and Beads entries

```
::: {.cell}
```

```
MC_Parameters <- markernames(MC_GatingSet)
MC_MarkersInterest <- MC_Parameters[!str_detect("Barcode|Beads", MC_Parameters)]
MC_MarkersInterest
```

```
:::
```

Dy161Di

“H2BK120Ub-macroH2A” Dy162Di “CrotonylK-H3K4me3” Dy163Di “H3R2cit-H2A.Z”
Dy164Di “H3K14ac-H3K36me1” Er166Di “CD33” Er167Di “CD16” Er168Di “H4K16ac-

H4K20me1" Er170Di "CD3" Eu151Di "H3K23ac-H3K27me1" Eu153Di "H2BS14ph-H3K36me2" Gd155Di "CD11c" Gd156Di "H3K18ac-H4K20me2" Gd158Di "H3K56ac-H3.3" Gd160Di "PADI4-H4K20me3" Ho165Di "H33Xcit-H3K27me3" Ir191Di "DNA" Ir193Di "DNA" Lu175Di "CD19" Lu176Di "beads" Nd142Di "gammaH2AX-Rme1" Nd143Di "H2BK5ac-Rme2sym" Nd144Di "H3S10ph-H3K4me2" Nd145Di "CD4" Nd146Di "CD8" Nd148Di "CD34" Nd150Di "H3.3S31ph-H3K36me3" Pr141Di "H3" Pt195Di "livedead" Sm147Di "H4K5ac-H3K9me2" Sm149Di "CleavedH3T22-H3K9me1" Sm152Di "H3K9ac-Rme2asy" Sm154Di "H2AK119Ub-H3K27me2" Tb159Di "CD14" Tm169Di "CD123" Y89Di "CD45" Yb171Di "CD38" Yb172Di "CD56" Yb173Di "H4" Yb174Di "H3K27ac-CENPA" Yb176Di "HLADR"

Of which, we want the names for this named list. So....

```
InterestingMetalsOnly <- names(MC_MarkersInterest)
InterestingMetalsOnly
```

```
[1] "Dy161Di" "Dy162Di" "Dy163Di" "Dy164Di" [5] "Er166Di" "Er167Di" "Er168Di"
"Er170Di" [9] "Eu151Di" "Eu153Di" "Gd155Di" "Gd156Di" [13] "Gd158Di" "Gd160Di"
"Ho165Di" "Ir191Di" [17] "Ir193Di" "Lu175Di" "Lu176Di" "Nd142Di" [21] "Nd143Di"
"Nd144Di" "Nd145Di" "Nd146Di" [25] "Nd148Di" "Nd150Di" "Pr141Di" "Pt195Di"
[29] "Sm147Di" "Sm149Di" "Sm152Di" "Sm154Di" [33] "Tb159Di" "Tm169Di" "Y89Di"
"Yb171Di" [37] "Yb172Di" "Yb173Di" "Yb174Di" "Yb176Di"
```

With these, we can then combine them with our transformer of interest to create the `transformerList()` that we can then apply to the samples.

```
Asinh <- flowjo_fasinh_trans()
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
str(MyAsinhTransform[1:3])
```

- `str(MyAsinhTransform[1:3])` List of 3 \$ Dy161Di:List of 9 ..\$ name : chr "flowJo_fasinh"
 - ..\$ transform :function (x)
 - ..\$ inverse :function (x)
 - ..\$ d_transform : NULL ..\$ d_inverse : NULL ..\$ breaks :function (x)
 - ..\$ minor_breaks:function (b, limits, n)
 - ..\$ format :function (x)
 - ..\$ domain : num [1:2] -Inf Inf ..- attr(, "class")= chr "transform" \$ Dy162Di:List of 9
 - ..\$ name : chr "flowJo_fasinh" ..\$ transform :function (x)
 - ..\$ inverse :function (x)
 - ..\$ d_transform : NULL ..\$ d_inverse : NULL ..\$ breaks :function (x)
 - ..\$ minor_breaks:function (b, limits, n)
 - ..\$ format :function (x)
 - ..\$ domain : num [1:2] -Inf Inf ..- attr(, "class")= chr "transform" \$ Dy163Di:List of 9

```

..$ name : chr "flowJo_fasinh" ..$ transform :function (x)
..$ inverse :function (x)
..$ d_transform : NULL ..$ d_inverse : NULL ..$ breaks :function (x)
..$ minor_breaks:function (b, limits, n)
..$ format :function (x)
..$ domain : num [1:2] -Inf Inf ..- attr(*, "class")= chr "transform"
transform(MC_GatingSet, MyAsinhTransform)

```

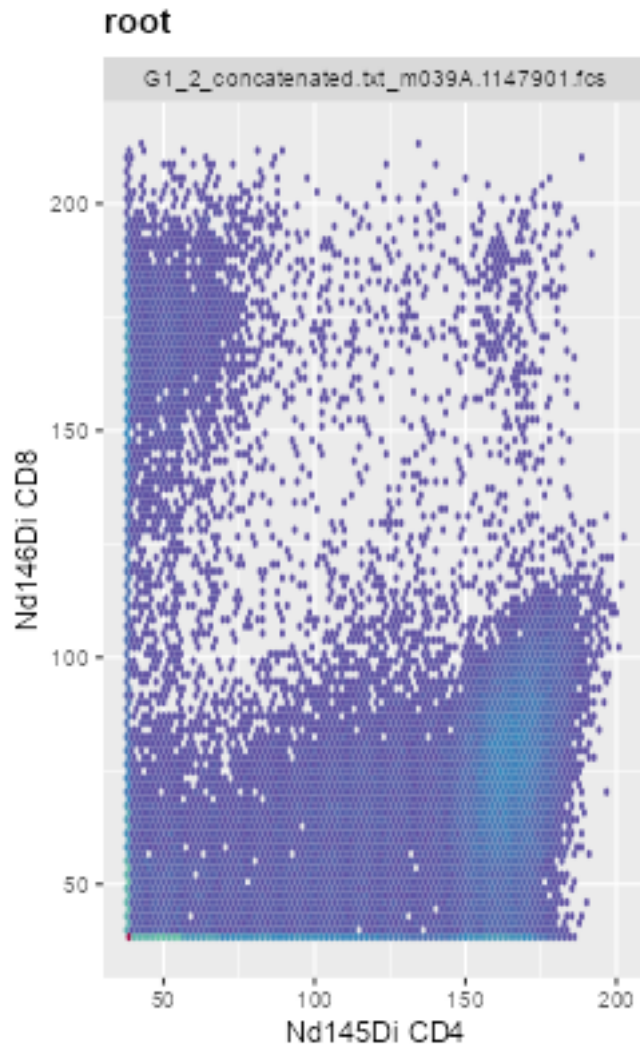
A GatingSet with 3 samples

With this done, let's try replotting the previous CD4 by CD8 plot

```

ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)

```



Asinh Arguments Similar to the case with SFC, we may need to revise some of the values provided to the individual arguments. Lets similarly investigate what we can do.

```
:: { .cell }
```

```
?flowjo_fasinh_trans
```

```
::
```

Which turns out to be another wrapper. Lets try the similarly named function

```
:: { .cell }
```



```
?flowjo_fasinh
```

```
:::
```

As we may remember, our current setting are the following

```
::: {.cell}
```

```
MC_cytoSet <- load_cytoSet_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoSet)
Asinh <- flowjo_fasinh_trans(m = 4, t = 12000, a = 0.7, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

```
:::
```

A GatingSet with 3 samples

```
::: {.cell}
```

```
gh_get_transformations(MC_GatingSet[[1]])$`Nd145Di`
```

```
:::
```

(a serial of codes that I erased because of problems rendering with latex)

```
<bytecode: 0x000001d6fc828e28> <environment: 0x000001d6f7d23dd0> attr(,"type") [1]
"fasinh"
```

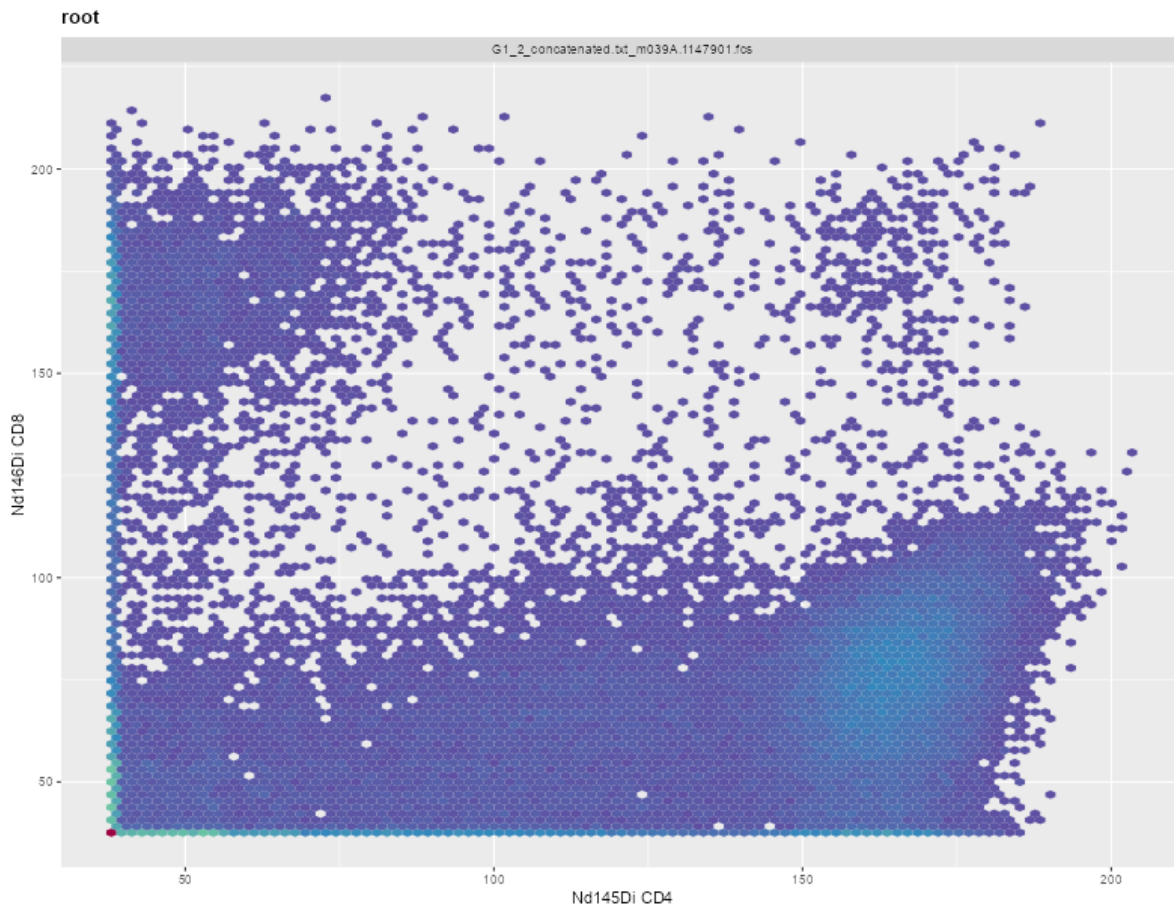
m

Being completionist, lets change the MC arguments as well. M is width in asymptotic decades, currently set to 4.

```
MC_cytoSet <- load_cytoSet_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoSet)
Asinh <- flowjo_fasinh_trans(m = 4, t = 12000, a = 0.7, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```



Lets set it to 5!

```
::: {.cell}
```

```
MC_cytoSet <- load_cytoSet_from_fcs(MC_files,
truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoSet)
Asinh <- flowjo_fasinh_trans(m = 5, t = 12000, a = 0.7, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

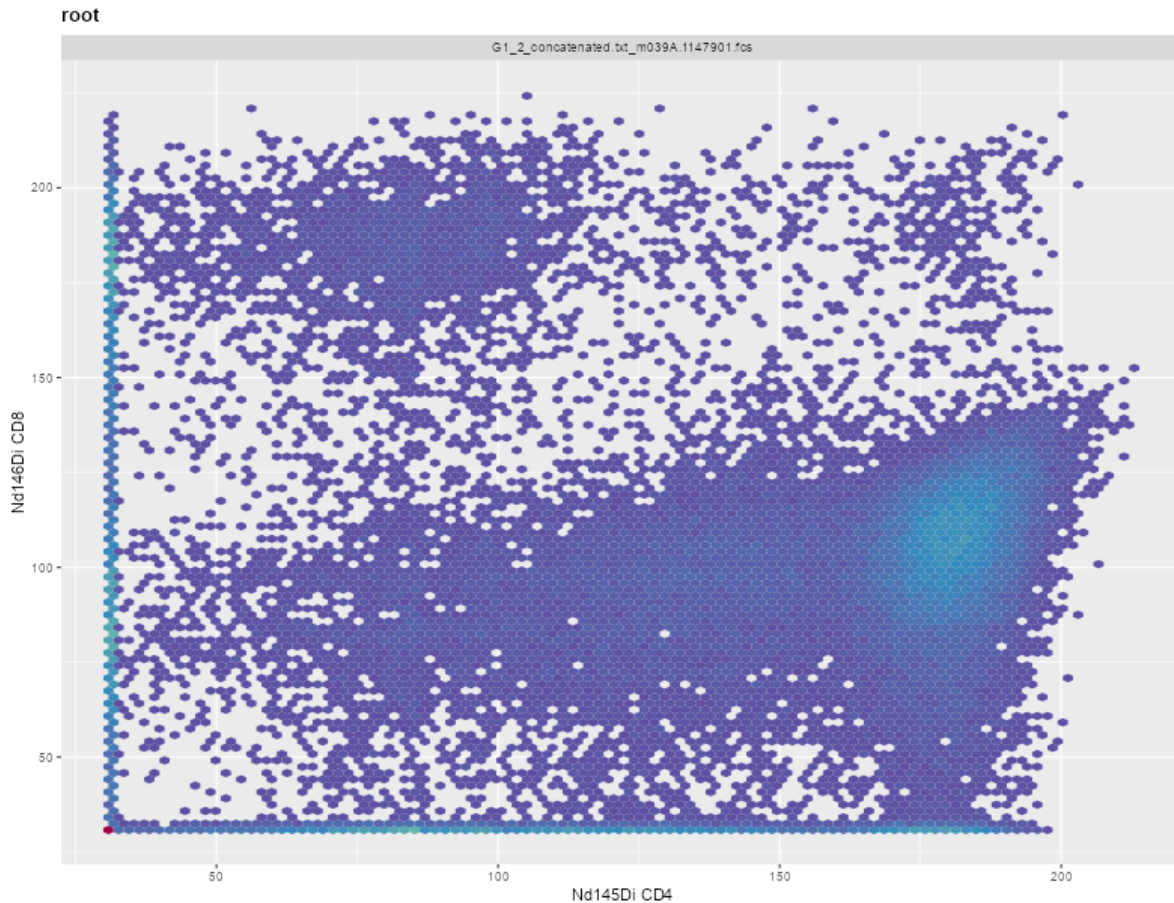
```
:::
```

A GatingSet with 3 samples

```
::: {.cell}
```

```
ggcyto(MC_GatingSet[1], subset="root",  
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```

```
:::
```

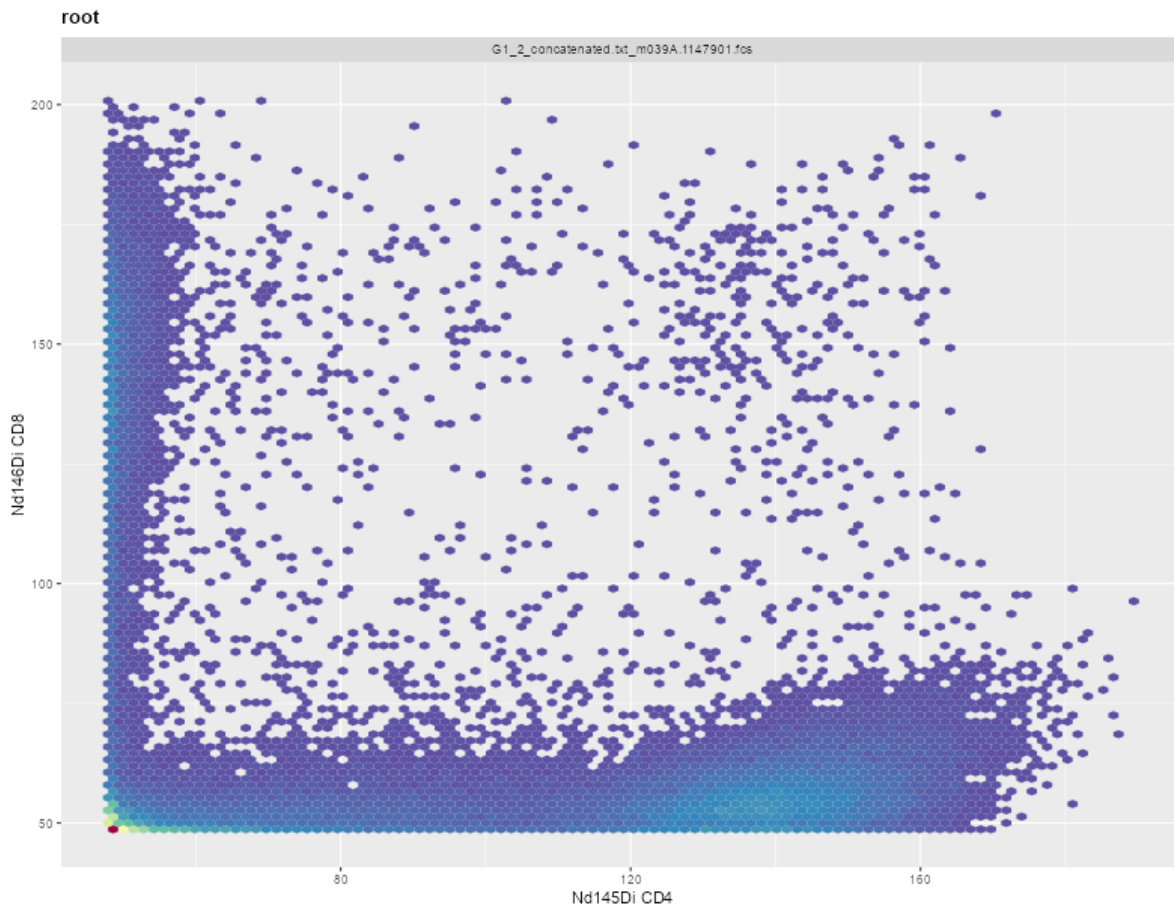


Vs. when we set it to 3

```
MC_cytoset <- load_cytoset_from_fcs(MC_files,  
  truncate_max_range = FALSE, transformation = FALSE)  
MC_GatingSet <- GatingSet(MC_cytoset)  
Asinh <- flowjo_fasinh_trans(m = 3, t = 12000, a = 0.7, length = 256)  
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)  
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```



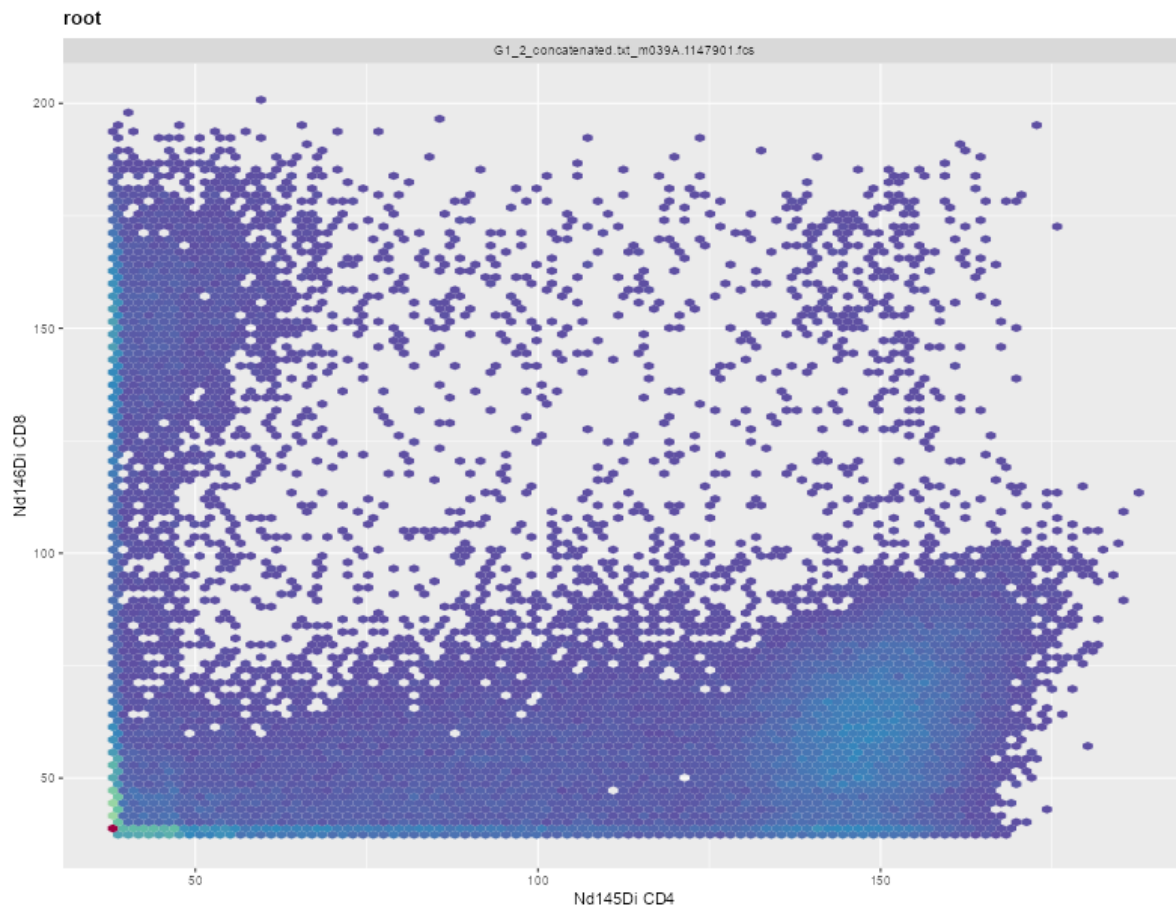
t

Increase t to 24000

```
MC_cytoSet <- load_cytoSet_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoSet)
Asinh <- flowjo_fasinh_trans(m = 4, t = 24000, a = 0.7, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```

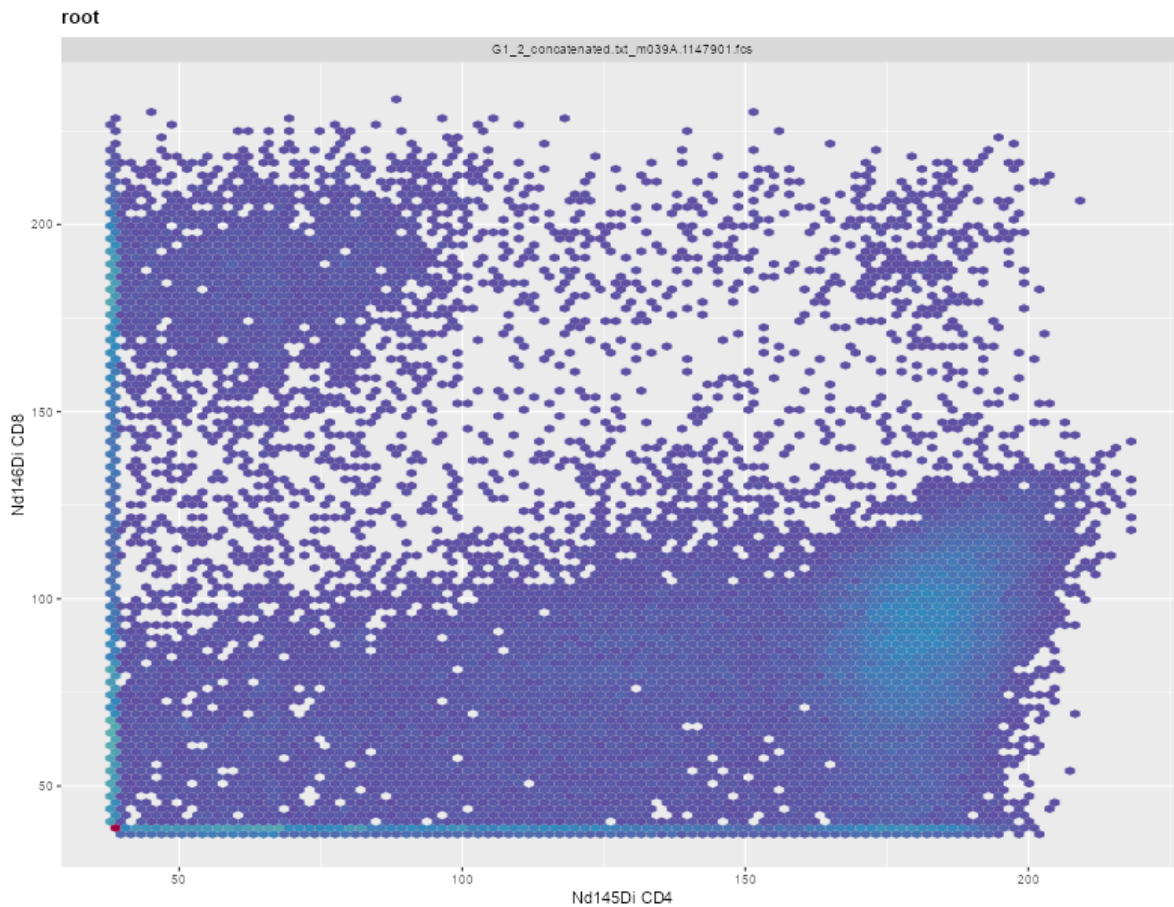


Decrease t to 6000

```
MC_cytoset <- load_cytoset_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoset)
Asinh <- flowjo_fasinh_trans(m = 4, t = 6000, a = 0.7, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples


```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```



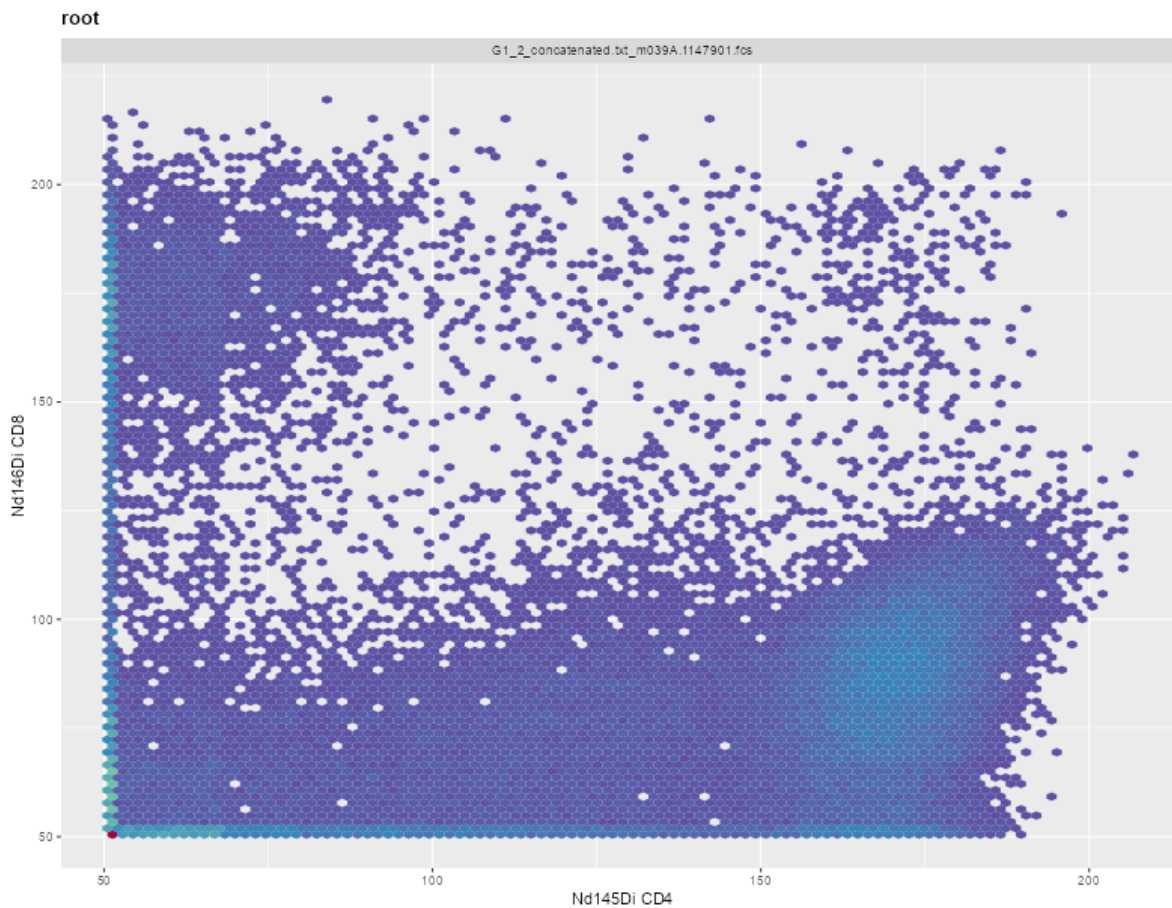
a

Increase negative decade to 1

```
MC_cytoset <- load_cytoset_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoset)
Asinh <- flowjo_fasinh_trans(m = 4, t = 12000, a = 1, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```

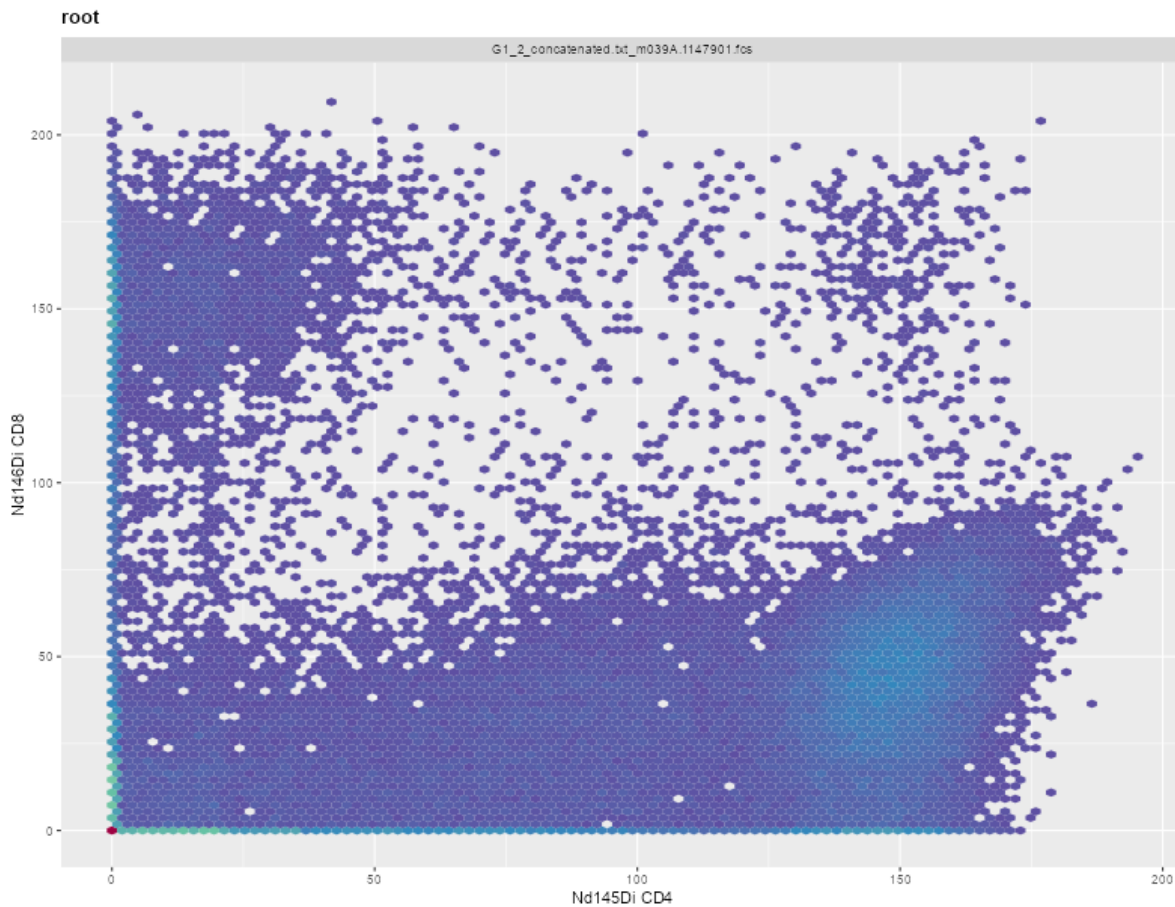


Vs. set to 0

```
MC_cytoset <- load_cytoset_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoset)
Asinh <- flowjo_fasinh_trans(m = 4, t = 12000, a = 0, length = 256)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```



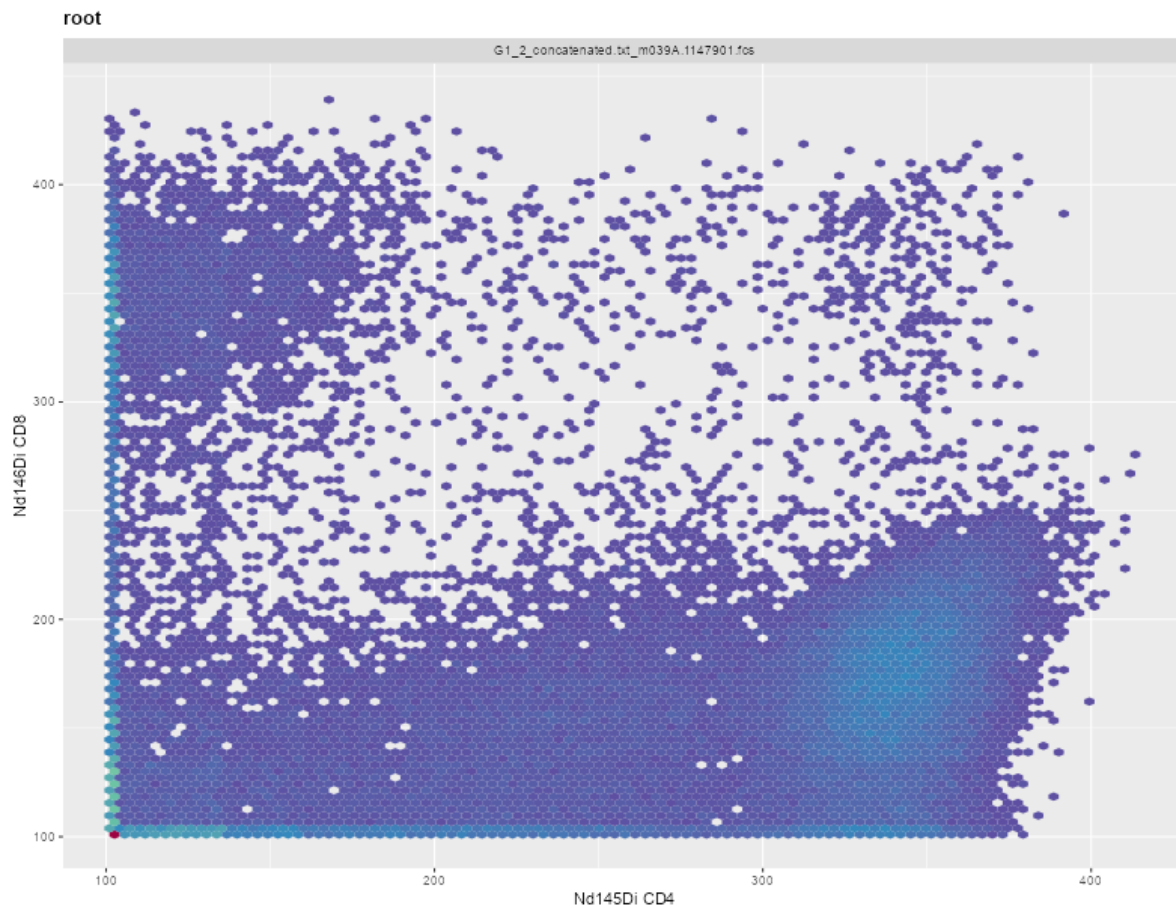
length

Doubling numeric max value of transformed data to 512

```
MC_cytoset <- load_cytoset_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoset)
Asinh <- flowjo_fasinh_trans(m = 4, t = 12000, a = 1, length = 512)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples


```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```

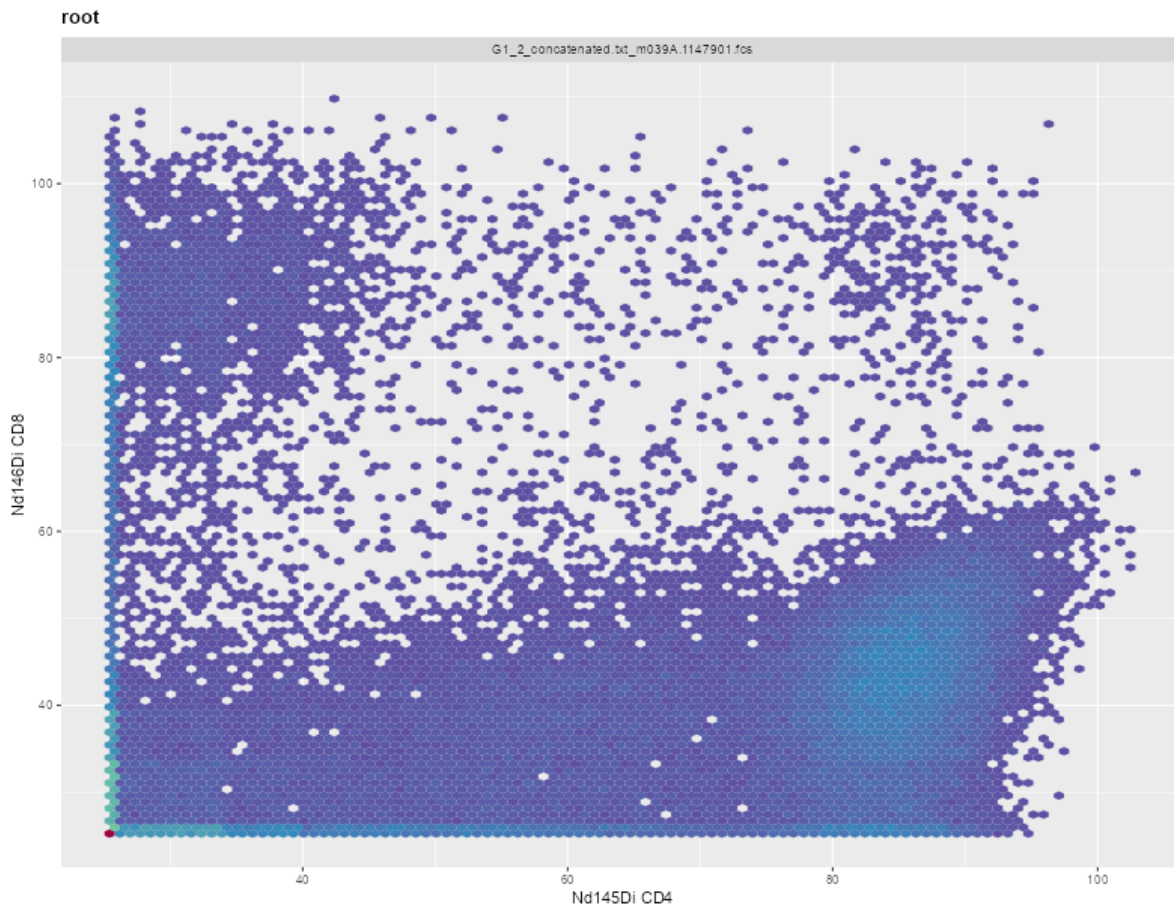


Vs. reducing it to 128

```
MC_cytoset <- load_cytoset_from_fcs(MC_files,
  truncate_max_range = FALSE, transformation = FALSE)
MC_GatingSet <- GatingSet(MC_cytoset)
Asinh <- flowjo_fasinh_trans(m = 4, t = 12000, a = 1, length = 128)
MyAsinhTransform <- transformerList(InterestingMetalsOnly, Asinh)
transform(MC_GatingSet, MyAsinhTransform)
```

A GatingSet with 3 samples

```
ggcyto(MC_GatingSet[1], subset="root",
aes( x="Nd145Di", y="Nd146Di")) + geom_hex(bins=100)
```



Extracting Data

One area where we will separately need to remember whether a transformation is applied or not is later on in the course, when we are trying to extract out the underlying exprs data for use in downstream analysis. If we wanted to do this for our current GatingSet, we first need to fetch the pointer contents to return a CytoSet object.

```
MyCytoSet <- gs_pop_get_data(SFC_GatingSet[[1]], "root")
MyCytoSet
```

A cytoset with 1 samples.

column names: Time, SSC-W, SSC-H, SSC-A, FSC-W, FSC-H, FSC-A, SSC-B-W, SSC-B-H, SSC-B-A, BUV395-A, BUV563-A, BUV615-A, BUV661-A, BUV737-A, BUV805-A, Pacific Blue-A, BV480-A, BV570-A, BV605-A, BV650-A, BV711-A, BV750-A, BV786-A, Alexa Fluor 488-A, Spark Blue 550-A, Spark Blue 574-A, RB613-A, RB705-A, RB780-A, PE-A, PE-Dazzle594-A, PE-Cy5-A, PE-Fire 700-A, PE-Fire 744-A, PE-Vio770-A, APC-A, Alexa Fluor 647-A, APC-R700-A, Zombie NIR-A, APC-Fire 750-A, APC-Fire 810-A, AF-A

cytoset has been subsetted and can be realized through `'realize_view()'`. We can then extract our the underlying MFI data using the `exprs()` function.

```
exprs(MyCytoSet[[1]])[1:3,]
```

Time	SSC-W	SSC-H	SSC-A
[1,] 396.1327 741155.4 495200 611700.2 [2,] 539.4743 730091.3 504832 614289.1 [3,] 590.1619 842852.1 267735 376101.7	FSC-W	FSC-H	FSC-A
[1,] 735140.7 1398530 1713527 715626.7 [2,] 714323.2 1607285 1913535 728604.2 [3,] 715528.4 824483 983235 781794.9	SSC-B-W	SSC-B-H	SSC-B-A
BUV395-A BUV563-A [1,] 416383 496624.7 3150.359 2315.777 [2,] 385314 467902.4 2064.067 3614.399 [3,] 309681 403511.7 3336.325 2874.082	BUV615-A	BUV661-A	BUV737-A
BUV805-A [1,] 2182.396 2132.781 2889.922 3597.062 [2,] 2124.969 1805.567 2639.037 1866.229 [3,] 2627.830 2043.606 3079.823 3621.227	Pacific Blue-A	BV480-A	BV570-A
[1,] 2228.710 1951.972 2101.419 [2,] 1999.236 2086.402 2219.585 [3,] 2317.032 2325.820 1490.307	BV605-A	BV650-A	BV711-A
BV750-A [1,] 3463.058 1927.428 2248.063 2060.087 [2,] 2549.794 3716.182 2128.070 2092.538 [3,] 3675.083 1788.000 1427.809 2477.396	BV786-A	Alexa Fluor 488-A	[1,] 2392.586 2060.505 [2,] 2267.093 2243.353 [3,] 1642.587 2500.961
Spark Blue 550-A Spark Blue 574-A [1,] 3532.579 3168.161 [2,] 3402.286 3383.590 [3,] 3638.622 3162.447	RB613-A	RB705-A	RB780-A
PE-A [1,] 2255.820 3850.380 2200.319 2602.714 [2,] 1700.675 2149.019 1946.190 2252.872 [3,] 1787.978 3751.768 2133.048 2754.971	PE-Dazzle594-A	PE-Cy5-A	PE-Fire 700-A
[1,] 2087.706 2216.431 2843.651 [2,] 2207.007 2933.298 3160.401 [3,] 2474.266 2275.351 2759.448	PE-Fire 744-A	PE-Vio770-A	APC-A
[1,] 2187.363 2112.620 1965.414 [2,] 2050.162 2190.069 2072.289 [3,] 2239.013 1918.631 2536.166	Alexa Fluor 647-A	APC-R700-A	[1,] 2199.804 2264.987 [2,] 2058.767 2266.735 [3,] 1664.451 2332.929
Zombie NIR-A APC-Fire 750-A [1,] 2333.349 3265.242 [2,] 2252.724 2853.149 [3,] 2060.233 3339.516	APC-Fire 810-A	AF-A	[1,] 3262.465 2913.750 [2,] 2093.584 2897.793 [3,] 3174.300 2227.430

If you look at the data for a while, you may notice the MFI values seem abnormally abbreviated. This is because the values present were themselves transformed. If we had wanted to retrieve the raw, untransformed values, we would have needed to provide the `inverse.transform = TRUE` argument when fetching the CytoSet. This would have triggered the `inverse.transformation` stored within the `transformerList`, reverting values back to the original.

```
MyInversedCytoSet <- gs_pop_get_data(SFC_GatingSet[[1]],
"root", inverse.transform=TRUE)
exprs(MyInversedCytoSet[[1]])[1:3,]
```

Time	SSC-W	SSC-H	SSC-A
[1,]	396.1327	741155.4	495200
[2,]	539.4743	730091.3	504832
[3,]	590.1619	842852.1	267735
[1,]	735140.7	1398530	1713527
[2,]	715626.7	714323.2	1607285
[3,]	715528.4	824483	983235
[1,]	416383	496624.7	9453.96973
[2,]	385314	467902.4	50.15964
[3,]	309681	403511.7	17222.40234
[1,]	422.8893	265.4719	4429.869
[2,]	240.8778	-778.2961	2294.426
[3,]	2228.7263	-13.7176	7624.175
[1,]	572.5627	-300.9588	166.9555
[2,]	-152.3773	119.9522	542.8082
[3,]	869.9310	900.6817	-2103.8748
[1,]	26514.346	-378.7812	636.1760
[2,]	1814.396	65177.4258	250.6365
[3,]	56177.691	-838.5793	-2473.9214
[1,]	37.73475	1143.6769	39.03822
[2,]	139.14844	699.4824	620.62848
[3,]	1483.97839	-1383.3655	1586.81445
[1,]	33800.78	9992.747	21516.37
[2,]	49274.09	9816.074	RB613-A
[3,]	-1154.0510	316.7392	-319.2414
[1,]	2087.5488	124.0298	532.5582
[2,]	652.0997	502.0220	4992.3154
[3,]	3095.6575	1470.6257	727.2120
[1,]	9753.685	6.749563	447.4721
[2,]	3131.895	606.339661	-406.8256
[3,]	75.84125	33.61102	698.2842
[1,]	1748.74023	-1294.83301	925.7467
[2,]	651.61072	4010.373	38.18906
[3,]	10186.6602	568.3808	

Right now, this won't affect much, but we will encounter where this matters in a couple weeks when we discuss how to visualize fluorescent signatures, and their subsequent use in unmixing.